Computer Networks 282 (2026) 112267

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

Joint caching-trajectory optimization for dynamic UAV-MEC networks: Context-aware game combined MADDPG

Zilin Wang [a], Liming Chen [b], Pengxian Chen [a], Yifeng Zheng [a], Wenjie Zhang [a,*]

[a] Key Laboratory of Data Science and Intelligence Application, School of Computer Science, Minnan Normal University, 363000, Zhangzhou, Fujian, China
[b] Zhangzhou Affiliated Hospital, Fujian Medical University, 363000, Zhangzhou, Fujian, China

ARTICLE INFO

Keywords:
Mobile edge computing (MEC)
Unmanned aerial vehicle(UAV)
Dynamic trajectory
Service caching
Resource allocation
Multi-Agent deep deterministic policy gradient (MADDPG).

ABSTRACT

Despite the growing attention toward Unmanned Aerial Vehicle (UAV)-assisted Mobile Edge Computing (MEC), the inherent energy constraints of UAVs and the requirement for low task delay remain significant challenges for its practical application. This paper investigates the joint optimization of task offloading, service caching, trajectory planning and resource allocation in the multi-UAV-assisted MEC system to minimize long-term average execution delay. We formulate the problem as a Mixed-Integer Nonlinear Programming (MINLP) problem and propose a Context-Aware Potential-Game-based Multi-Agent Deep Deterministic Policy Gradient (CAPG-MADDPG) algorithm to tackle the coupled discrete-continuous optimization and complex action space. The original problem is decoupled into three subproblems. Specifically, the task offloading subproblem is solved using Potential Game (PG) algorithm; then, the service caching subproblem is addressed by Contextual Combinatorial Multi-Armed Bandit (CCMAB) algorithm; finally, the UAV trajectory planning and resource allocation subproblem is jointly optimized via MADDPG algorithm. Experimental results demonstrate that the proposed CAPG-MADDPG algorithm outperforms other Deep Reinforcement Learning (DRL) and caching algorithms in terms of long-term completion delay, while also exhibiting fast convergence and better training stability.

1. Introduction

The development of 5G technology has led to an increase in computation-intensive and latency-sensitive applications, such as Virtual Reality (VR) and Augmented Reality (AR) [1]. These applications impose significant computational and real-time processing demands on resource-constrained terminal devices [2,3]. While cloud center offers substantial computational power and coverage, this processing mode requires tasks to be transmitted to remote data centers, introducing high communication latency and increased operating costs. In contrast, Mobile Edge Computing (MEC) is a distributed computing paradigm that deploys resources at network edge (e.g., base stations) located close to users. This approach optimizes resource allocation and offloading decision to reduce transmission distance and minimize latency [4]. However, conventional MEC architectures predominantly rely on fixed-location base stations, which exhibit limited service coverage and flexibility. Consequently, these systems are less capable of adapting to dynamic network environments and time-varying user task demands [5].

Unmanned Aerial Vehicles (UAVs) provide flexibility and mobility, enabling rapid deployment in dynamic environments [6]. By offering on-demand services, UAVs can effectively mitigate the limitations of conventional MEC in challenging scenarios like remote mountainous regions and dense urban areas [7], while their proximity to users enables low-latency, high-quality Line-of-Sight (LoS) links [8]. Owing to these distinctive advantages, UAV-assisted MEC has attracted considerable research interest in recent years [9]. However, due to the inherent constraints in computation, storage, and energy, the resource allocation and task offloading in UAV-assisted MEC remain highly challenging. To address this issue, existing studies have focused on single-UAV systems, employing techniques such as Deep Reinforcement Learning (DRL) [10,11] to minimize latency and energy consumption. Nevertheless, single UAV has limited coverage and processing capability, which may result in high latency under heavy task loads or numerous users. In contrast, multi-UAV collaborative compution can better fulfill user demands while improving resource utilization. Studies have explored distributed offloading strategies [12] and multi-agent reinforcement learning approaches [13] to coordinate these systems. Despite these advancements, due to significant limitations in computation and battery storage, the optimal allocation of limited resources in UAV-assisted MEC systems is still an open problem.

* Corresponding author.
E-mail addresses: g2024061015@stu.mnnu.edu.cn (Z. Wang), 282073074@qq.com (L. Chen), 2228950778@qq.com (P. Chen), zyf@mnnu.edu.cn (Y. Zheng), zhan0300@ntu.edu.sg (W. Zhang).

https://doi.org/10.1016/j.comnet.2026.112267
Received 31 December 2025; Received in revised form 27 February 2026; Accepted 26 March 2026
Available online 27 March 2026
1389-1286/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Service caching has emerged as an effective approach to alleviate the resource constraints of UAVs. By pre-storing popular services on UAV-mounted edge servers, single UAV can improve response latency and resource utilization [14]. To achieve further gains, multi-UAV collaborative service caching strategies have been proposed, employing methods such as reinforcement learning [15] and heuristic algorithms [16]. Nevertheless, these strategies often fail to consider UAVs mobility, which limits the potential gains in overall system performance. Dynamically adjusting UAV trajectories towards areas with high user density can improve service cache hit rates [17]. Thus, the joint optimization of task offloading, UAV trajectory, and service caching has been proposed to expand the coverage and improve network performance [18]. Despite these efforts, several key challenges remain due to the incomplete consideration of mobility, energy, and environmental dynamics: (1) *Demand uncertainty:* In dynamic network environments, user requests are stochastic and time-varying. Applications like AR/VR involve unpredictable interactions, leading to rapid and real-time fluctuations in service demands. This unpredictability complicates service caching and resource allocation strategies. (2) *Channel dynamics:* The quality of air-ground channel is highly susceptible to the distance between UAVs and Ground Users (GUs). Shorter distances increase the likelihood of LoS communication. However, UAV mobility causes continuous channel fluctuations, which adversely impacts data transmission reliability and efficiency. (3) *Energy queue stability:* UAVs typically operate on limited onboard battery capacity. Maintaining an energy balance is critical to ensure sustained service stability.

Existing studies tend to focus on specific combinations of service caching, task offloading, trajectory planning and computing resource allocation. They rarely investigate the joint optimisation of these factors in dynamic, multi-UAV-assisted MEC systems under energy constraints. To address these challenges, this paper investigates a multi-UAV-assisted air-ground MEC network. Each UAV is equipped with computing and caching units, as well as Energy Harvesting (EH) module. With the aim to minimize the long-term average delay, this paper jointly optimizes UAV trajectory, resource allocation, service caching, and task offloading under the consideration of demand uncertainty, channel dynamics, and energy stability. This problem is formulated as a Mixed-Integer Nonlinear Programming (MINLP), which is computationally intractable due to highly coupling between discrete and continuous variables and the time-varying network environment. We propose a Context-Aware Potential-Game-based Multi-Agent Deep Deterministic Policy Gradient algorithm (CAPG-MADDPG) by decomposing the original problem into three subproblems. For the service caching subproblem, a Contextual Combinatorial Multi-Armed Bandit (CCMAB) algorithm is employed to improve cache hit rates based on historical request patterns and contextual information. Meanwhile, the task offloading subproblem is modeled using Potential Game (PG), enabling the design of a distributed algorithm that converges to a Nash equilibrium. The trajectory planning and resource allocation subproblem is addressed by multi-agent deep deterministic policy gradient (MADDPG) algorithm, which employs a centralized-training-decentralized-execution paradigm to manage inter-UAV cooperation and competition in continuous action spaces. The main contributions of this paper are as follows:

The main contributions of this paper are as follows:

Joint trajectory and resource optimisation via MADDPG: We develop an MADDPG-based framework for the joint optimisation of UAV trajectories and resource allocation in dynamic environments. By learning continuous multi-agent control policies, this approach facilitates cooperative multi-agent learning, enabling real-time adaptation to channel variations and user demands, which improves scheduling efficiency and reduces latency.

Context-aware distributed service caching: We propose a CCMAB algorithm to handle stochastic and context-dependent user demands. The service caching decision is formulated as a discrete combinatorial optimisation problem and updated periodically based on observed execution outcomes. By proactively caching services based on learned patterns, the algorithm can improve the cache hit rate and service delivery efficiency.

Task offloading via PG: We model the multi-UAV task offloading problem as a PG, which captures the strategic coupling among multiple UAVs and GUs, design a distributed decision-making mechanism, and leverage a strictly constructed potential function to prove convergence to a Nash Equilibrium. This formulation enables efficient distributed optimisation and effectively reduces the training complexity for individual UAV agent.

Performance evaluation: Simulation results demonstrate that the proposed coordinated framework integrating CCMAB-based service caching, PG-based task offloading, and MADDPG-based trajectory control minimizes the system's long-term average delay while ensuring the long-term stability of UAV energy queues.

2. Related work

2.1. Task offloading and resource allocation

Task offloading and resource allocation problem has often been jointly optimized to reduce energy consumption and latency for MEC systems. Lin et al. [19] proposed a two-level alternating framework for single base station to minimize total energy consumption by jointly optimizing task offloading and resource allocation. Moreover, the work in [20] introduced a fairness-aware task offloading and resource allocation mechanism for multi-user multi-base station scenario. Despite these improvements, fixed MEC infrastructures have limited coverage and flexibility, making them inadequate for emergency communications and other unpredictable circumstances. UAV-assisted MEC has gained significant research attention in recent years. For example, in [10], an adaptive task offloading and resource allocation framework for single-UAV multi-user MEC was proposed to minimize long-term energy consumption and latency. However, single UAV system is difficult to support service demands in highly dynamic and large-scale environments. Consequently, multi-UAV-assisted MEC systems have attracted growing interest [12,13,21]. In [21], a software-defined network-enhanced multi-UAV collaborative MEC system with an optimization resource allocation scheme was developed to minimize task processing latency and balance energy consumption among UAVs. Similarly, in [12], the task offloading and resource allocation problem was modeled as a Semi-Markov Decision Process (Semi-MDP) to maximize long-term system utility. The authors in [13] constructed a profit-aware UAVs collaborative offloading framework to enhance task processing efficiency and improve resource utility. Although these studies have advanced task offloading and resource allocation in UAV-assisted MEC, mostly assume that UAVs can provide all required services. This assumption overlooks practical storage constraints and fails to consider the critical coupling relationship between task offloading and service caching.

2.2. Service caching

Caching popular services in advance at MEC nodes can alleviate backhaul pressure and reduce task response latency. In [22], a dynamic joint optimization algorithm based on random approximation was proposed to collaboratively optimize service caching and task offloading across the cloud and ground base stations. In [23], a location- and preference-driven caching strategy was developed for computation-intensive tasks to reduce system energy consumption. The work in [24] introduced a hierarchical edge caching mechanism based on popularity and game theory to improve cache hit rates and reduce latency by predicting user content demand. Furthermore, Luo et al. [25] developed an intelligent caching mechanism that clustered users based on content preferences to predict the preferred services, thereby minimizing task request and transmission delay. Zheng et al. [16] presented a caching strategy based on task latency priorities to minimize the maximum completion delay. Taimoor et al. [26] proposed a joint communication, com-

puting, caching, and control resource allocation framework based on the interior point method to minimize network latency and cost. Recently, Gao et al. [27] studied the joint optimization of task offloading, service caching, resource allocation and UAV trajectory planning for multi-UAV MEC systems with caching capabilities, aiming to maximize service-experience ratio via a centralized alternating optimization framework. However, most existing studies rely on static popularity models or periodic update strategies, which lack the adaptability required in dynamic multi-UAV-assisted MEC scenarios with frequently changing user demands. Therefore, a user demand-driven dynamic caching mechanism is required to improve cache hit rates and service quality.

2.3. Conventional optimization for UAV assisted MEC

Jointly optimizing UAV trajectories and task offloading is essential for enhancing the overall performance for UAV-assisted MEC systems. In early research, most studies focused on single-UAV scenarios and employed model-based optimization approaches, including convex optimization, Lyapunov-based control, and game-theoretic frameworks. For example, Wang et al. [28] employed a Lyapunov drift-plus-penalty mechanism combined with greedy and convex approximation algorithms to jointly optimize offloading and UAV trajectory, minimizing long-term latency and energy consumption. Similarly, the authors in [29] applied a two-stage Stackelberg differential game to jointly optimize single-UAV trajectory, resource allocation, and task offloading. Furthermore, Chen et al. [30] introduced a two-level framework based on non-cooperative and Stackelberg game to jointly optimize task offloading and resource pricing, aiming to minimize energy consumption and maximize utility. In multi-UAV scenarios, collaborative optimization has also been studied. For instance, the work in [31] proposed an algorithm based on Block Coordinate Descent (BCD) and Successive Convex Approximation (SCA) to optimize task offloading, resource allocation, and UAV trajectories, reducing delay and energy consumption. In [32], a joint optimization algorithm combining convex optimization and game-theoretic strategies was developed to minimize task latency and energy consumption in multi-UAV scenarios. Although these studies achieved multidimensional optimization, the real-time dynamics of user demands and channel conditions, as well as service caching constraints have not been fully investigated. Moreover, the integration of energy harvesting mechanisms to address UAV energy limitations remains an insufficiently explored.

2.4. Intelligent optimization for UAV-assisted MEC

Various single-agent DRL approaches have been explored to address for joint optimization problems in UAV-assisted MEC, such as Deep Q-network (DQN) [33,34], Deep Deterministic Policy Gradient (DDPG) [10], Twin Delayed Deep Deterministic Policy Gradient (TD3) [35] and Soft Actor-Critic (SAC) [36]. Dehkordi et al. [33] developed a DQN-based algorithm to jointly optimize UAV trajectories and resource allocation to maximize task processing efficiency. Darchini-Tabrizi et al. [34] proposed a Rainbow DQN-based multi-UAV trajectory planning method, where the agent indirectly determined UAVs' next slot target positions by selecting served users and offloading ratios to minimize processing delay. In [10], DDPG was proposed to investigate the UAV trajectory and intelligent reflecting surface phase offset to maximize system energy efficiency. Zhao et al. [35] proposed a centralised TD3-based framework that jointly optimized collaborative computation offloading, wireless charging scheduling and UAV trajectories in multi-UAV-assisted MEC systems to maximize long-term system utility. Li et al. [36] employed a SAC-based algorithm to jointly optimize UAV trajectories, user association, and power control, thereby enhancing system energy efficiency and ensuring user fairness. Furthermore, Zhao et al. [37] developed a DRL-based algorithm to jointly optimize 3D UAV trajectory design, collaborative computation offloading, and resource allocation in multi-UAV-assisted MEC networks for emergency rescue to maximize long-term system utility. Single-agent DRL methods operate independently and are suited for single-UAV scenarios. However, they face limitations in handling cooperative tasks offloading involving multiple UAVs. To address this issue, multi-agent DRL frameworks have been introduced to enable cooperative policy learning in multi-UAV environments. In [38], a joint optimization framework based on MDP was studied, and a twin-delay MADDPG algorithm was proposed to optimize offloading strategies and multi-UAV trajectories to minimize delay and ensure service fairness. Li et al. [39] proposed an embedded multi-agent DRL algorithm to jointly optimize task offloading and UAV trajectories, to maximize energy efficiency. Similarly, in [40], an extended MADDPG algorithm was designed for UAV-assisted vehicular networks to ensure vehicle service quality and minimize task completion latency. Additionally, in [41], a MADDPG-based collaborative optimization mechanism was designed to jointly regulate UAV trajectories and user transmission power to minimize energy consumption and task delay. Despite these advances, existing DRL approaches still face challenges such as high computational overhead, low exploration efficiency, and difficulties in coordinating multidimensional decisions in multi-user, multi-UAV scenarios. In particular, the joint optimization of service caching, task offloading, and trajectory planning in UAV-assisted MEC systems is highly challenging due to the randomness of user demands, UAV mobility, and energy queue stability, which requires the design of more efficient and intelligent solution.

The related papers are summarized in Table 1. Our work differs from the previous studies in the following aspects:

(1) Unlike [12,13,21], which focus only on the joint optimization of task offloading and resource allocation, this work considers the close and highly coupled relationship with service caching to further enhance system performance.
(2) In comparison to static popularity modeling or periodic update strategies in [16,22–27], which predominantly adopt static popularity modeling or periodic update strategies, this paper designs a demand-driven dynamic caching mechanism that accounts for heterogeneous user demands in time-varying scenarios.
(3) While prior studies [28–32] have explored multi-UAV trajectory planning, the critical issue of limited onboard energy remains insufficiently addressed. This work incorporates RF energy harvesting, enabling UAVs to recharge from the wireless environment and provide long-term, stable services to users.
(4) In contrast to [33–41], the proposed CAPG-MADDPG algorithm not only leverages MADDPG to optimize UAV trajectories but also extends the optimization scope to include task caching, resource allocation, task offloading, and energy balance, thereby improving system performance.

3. System model

We consider a UAV-assisted MEC system consisting of M UAVs and N GUs, as illustrated in Fig. 1. The sets of UAVs and GUs are denoted by $\mathcal{M} = \{1, 2, \ldots, M\}$ and $\mathcal{N} = \{1, 2, \ldots, N\}$, respectively. Owing to limited computational resources, GUs are generally unable to execute computation-intensive tasks locally. Instead, these tasks are offloaded to the associated UAVs for processing. Each UAV pre-caches the relevant services from the cloud and subsequently positions itself strategically in close proximity to served GUs to enable low-latency service delivery. Notably, each UAV provides distinct services to GUs, introducing service heterogeneity across the aerial network. In this paper, J types of service requests are considered, denoted as $\mathcal{G} = \{g_1, g_2, \ldots, g_J\}$. Due to limited cache capacity, dynamic and demand-aware service caching strategies must be developed to adapt to user request variations. Furthermore, UAV trajectories should be optimized to reduce energy consumption, avoid inter-UAV collisions, and enhance the performance of UAV-assisted MEC network.

Table 1
Comparative Summary of Related Works.

Reference	Task offloading	Resource allocation	Service caching	Trajectory planning	Method
[12,13]	√	√	×	×	DRL
[16]	√	√	√	×	BCD + SCA
[21]	√	√	×	×	Convex + Game
[22]	×	×	√	×	Game + Randomized Rounding
[23]	√	√	√	×	Deep learning
[24]	×	√	√	×	Game + Backward induction
[25]	×	√	√	×	K-means + Q-learning
[26]	√	√	√	×	Interior point
[27]	√	√	√	√	SCA + CVX
[28,31]	√	√	×	√	SCA
[29,30]	√	√	×	×	Game
[32]	√	√	×	×	Game + Genetic algorithm
[33,34,39]	√	√	×	√	DQN
[35]	√	√	×	√	TD3
[36]	×	√	×	√	SAC + MDP
[37]	√	√	×	√	DRL
[38]	√	√	×	√	MATD3
[40,41]	√	√	×	√	MADDPG
Proposed	√	√	√	√	CAPG-MADDPG

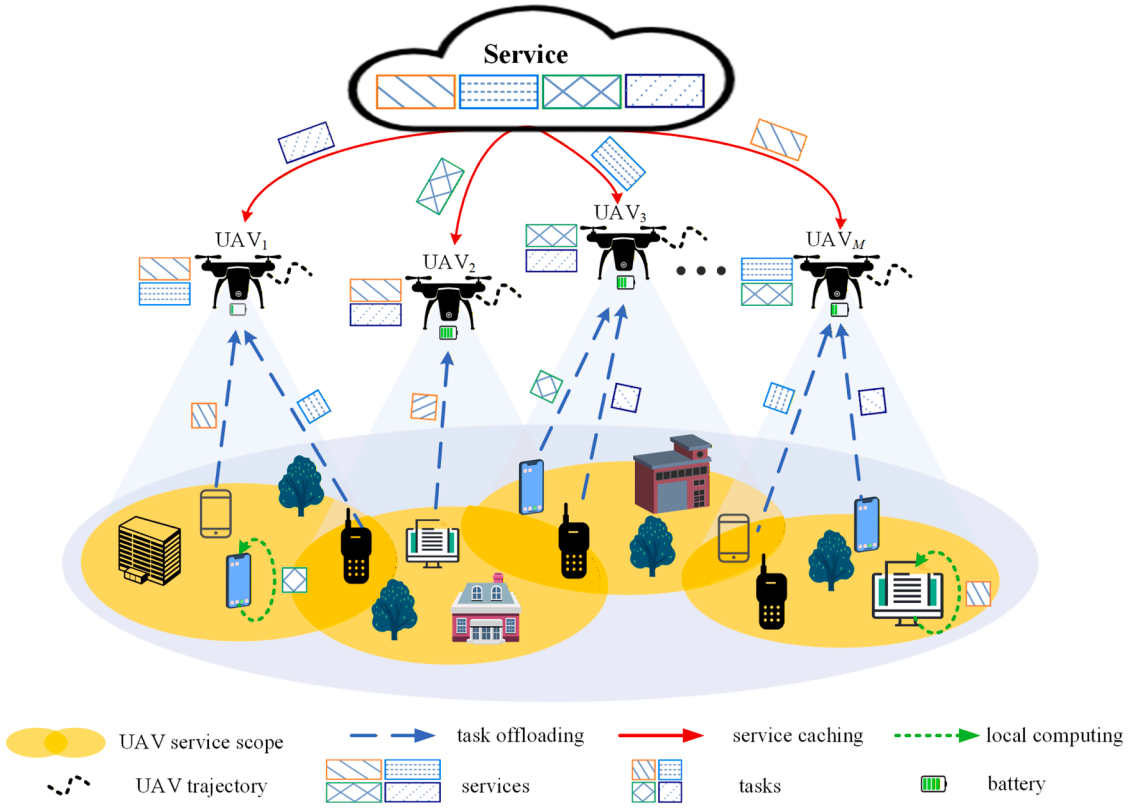


Fig. 1. Multi-UAV-assisted MEC task offloading and service caching model.

The operational timeline is discretized into multiple equal-length periods. Each period is further partitioned into T uniform time slots, denoted as $\Gamma = \{1, 2, \ldots, \tau, \ldots, T\}$. Within time slot τ, each GU_n generates a computational task $U_n(\tau)$, characterized by the tuple $\{I_n(\tau), C_n(\tau), T_n(\tau), g_n(\tau)\}$. Here, $I_n(\tau)$ denotes the input data size, $C_n(\tau)$ indicates the required number of CPU cycles to process the task, $T_n(\tau)$ is the maximum tolerable latency, and $g_n(\tau) \in \mathcal{G}$ specifies the type of service requested. Channel conditions, user demands and locations, UAV positions are assumed to remain constant within a time slot but may vary across different time slots. These time-varying conditions will affect data transmission and caching efficiency. Furthermore, due to the unpredictable and dynamic nature of user requests, both service caching and task offloading strategies are required to adaptively update at the beginning of each time slot. Meanwhile, UAVs trajectories need dynamic optimization to improve coverage and service availability. This leads to a complex joint optimization problem involving UAV trajectory scheduling, task offloading, and service caching, subject to constraints including limited onboard energy, time-varying channel conditions, and highly dynamic user demands.

3.1. UAV mobility model

The UAVs are assumed to fly at a fixed altitude h. During time slot τ, the coordinate of UAV_m is denoted by $\mathbf{w}_m^U(\tau) = [x_m^U(\tau), y_m^U(\tau), h]$, where $x_m^U(\tau)$, $y_m^U(\tau)$, and h represent its positions along the X, Y, and Z axes, respectively. The set of all UAV positions is defined by

$\mathbf{W}^U(\tau) = \{\mathbf{w}_1^U(\tau), \ldots, \mathbf{w}_M^U(\tau)\}$. The mobility of each UAV is constrained by maximum flight speed $v_{\max}$, which limits its possible positions between consecutive time slots, that is:

$$\left| |\mathbf{w}_m^U(\tau+1) - \mathbf{w}_m^U(\tau)| \right| \le v_{max}\Delta\tau, \forall m, \tag{1}$$

where $\Delta\tau$ denotes the duration of time slot τ.

To ensure collision-free operations among multi-UAV, a minimum safety inter-UAV distance $D^s_{\min}$ must be maintained, leading to following constraint:

$$\left\| \mathbf{w}_m^U(\tau) - \mathbf{w}_{m'}^U(\tau) \right\| \ge D^s_{min}, \forall m \ne m'. \tag{2}$$

The movement of a UAV depends on its azimuth angle $\theta_m \in [0, \pi]$ and flight speed $v_m \in [0, v_{\max}]$. Let $\Delta\theta_m[\tau]$ and $\Delta v_m[\tau]$ denote the incremental changes in the azimuth angle and flight speed for UAV_m in time slot τ, the evolutions of these states are governed by the update equations:

$$\begin{cases} \theta_m(\tau) = \theta_m(\tau-1) + \Delta\theta_m(\tau), \\ v_m(\tau) = v_m(\tau-1) + \Delta v_m(\tau). \end{cases} \tag{3}$$

Thus, the update equations for UAV trajectory between adjacent time slots are given by:

$$\begin{cases} x_m^U(\tau+1) = x_m^U(\tau) + v_m(\tau)\cos\theta_m(\tau)\Delta\tau, \\ y_m^U(\tau+1) = y_m^U(\tau) + v_m(\tau)\sin\theta_m(\tau)\Delta\tau. \end{cases} \tag{4}$$

3.2. Communication model

The primary communication mode is Ground-to-Aerial (G2A) transmission from GU to UAV. Since the size of computational result is usually much smaller than the input data, the downlink transmission can be ignored. To accurately model G2A communication, a probabilistic path loss model which accounts for both LoS and Non-Line-of-Sight (NLoS) communication is introduced [42]. Let $\mathbf{w}_n^A(\tau) = [x_n^A(\tau), y_n^A(\tau), 0]$ denote the coordinates of GU_n, then, the distance between GU_n and UAV_m is expressed as:

$$d_{n,m}(\tau) = \left| |\mathbf{w}_n^A(\tau) - \mathbf{w}_m^U(\tau)| \right|. \tag{5}$$

The probabilities of LoS and NLoS communication in G2A transmission, denoted as $Pr_{n,m}^{LoS}(\tau)$ and $Pr_{n,m}^{NLoS}(\tau)$, are defined as [42]:

$$Pr_{n,m}^{LoS}(\tau) = \frac{1}{1 + \alpha e^{-\beta\left(\left(\frac{180}{\pi}\right)\arcsin\left(\frac{h}{d_{n,m}(\tau)}\right) - \alpha\right)}}, \tag{6}$$

$$Pr_{n,m}^{NLoS}(\tau) = 1 - Pr_{n,m}^{LoS}(\tau), \tag{7}$$

where α and β are environment-dependent constants.

Let V_c denote the speed of light and f_c is the carrier frequency, then the free-space path loss, $PL_{n,m}(\tau)$ can be calculated by:

$$PL_{n,m}(\tau) = 20\lg\left(\frac{4\pi}{V_c}\right) + 20\lg\left(f_c\right) + 20\lg\left(d_{n,m}(\tau)\right). \tag{8}$$

The path loss for both LoS/NLoS links $LS_{n,m}^{\varphi}(\tau)$ is:

$$LS_{n,m}^{\varphi}(\tau) = PL_{n,m}(\tau) + PL_{\varphi}, \tag{9}$$

where PL_{φ} denotes the additional path loss and $\varphi \in \{LoS, NLoS\}$ represents the communication type. Then the average path loss and the corresponding channel gain from GU_n to UAV_m are:

$$\overline{LS_{n,m}}(\tau) = LS_{n,m}^{LoS}(\tau)Pr_{n,m}^{LoS}(\tau) + LS_{n,m}^{NLoS}(\tau)Pr_{n,m}^{NLoS}(\tau), \tag{10}$$

$$H_{n,m}(\tau) = \frac{1}{\overline{LS_{n,m}}(\tau)}. \tag{11}$$

The uplink transmission rate $R_{n,m}^{G2A}(\tau)$ from GU_n to UAV_m is given by:

$$R_{n,m}^{G2A}(\tau) = B\log_2\left(1 + \frac{p_n^{tr}(\tau)H_{n,m}(\tau)}{I_{n,m} + N_o}\right), \tag{12}$$

where B denotes the bandwidth, $p_n^{tr}(\tau)$ is the transmission power of GU_n, N_o denotes the noise power, and $I_{n,m}$ represents the interference from other GUs, defined as:

$$I_{n,m} = \sum_{n' \ne n} p_{n'}^{tr}(\tau)H_{n',m}(\tau). \tag{13}$$

To establish and maintain a reliable communication link between GU and UAV, the distance must not exceed the maximum communication range D_{G2A}. We use $\mathcal{N}_m(\tau) = \left\{ n \in \mathcal{N} \,\middle|\, d_{n,m}(\tau) \le D_{\text{G2A}} \right\}$ to denote the set of GUs associated with UAV_m. We further introduce binary variable $x_{n,m}(\tau) \in \{0, 1\}$ to represent the offloading decision of GU_n, where $x_{n,m}(\tau) = 1$ indicates that GU_n offloads task to the associated UAV_m, and $x_{n,m}(\tau) = 0$ otherwise. When multiple UAVs satisfy both the coverage constraint and service caching feasibility, GU_n selects the UAV that minimizes its predicted total task completion delay. Each GU can offload a task to at most one UAV and each UAV can serve at most $N_m^{\max}$ GUs, therefore, the following constraint must be satisfied:

$$\sum_{m=1}^{M} x_{n,m}(\tau) \le 1, \forall n. \tag{14}$$

$$\sum_{n=1}^{N} x_{n,m}(\tau) \le N_m^{max}, \forall m. \tag{15}$$

3.3. Caching model

At the beginning of each time slot, each UAV performs service caching according to the demands of associated GUs. Let $S(\tau) = \{S_m(\tau) | m \in \mathcal{M}\}$ denote the caching strategies of all UAVs in time slot τ, where $S_m(\tau) = \{s_{m,g_j}(\tau) \in \{0, 1\} | g_j \in \mathcal{G}\ge\}$ represents the service caching strategy of UAV_m. $s_{m,g_j}(\tau) = 1$ indicates that service g_j is cached on UAV_m during slot τ. The storage required by each service g_j is ϕ_{g_j}, then, the caching strategy must satisfy the following constraint:

$$\sum_{j=1}^{J} s_{m,g_j}(\tau)\phi_{g_j} \le \Phi_m^{max}, \tag{16}$$

where $\Phi_m^{\max}$ is the cache capacity of UAV_m.

GUs can only offload tasks to the associated UAV if the required services are cached. If no UAV caches the required service, the task is executed locally at the GU. This dependency is captured by the following constraint:

$$x_{n,m}(\tau) \le s_{m,g_n}(\tau), \forall m, n. \tag{17}$$

3.4. Delay model

Local computing: The total delay depends on the offloading decision. If a task is computed locally by GU_n (i.e., $\sum_{m=1}^{M} x_{n,m}(\tau) = 0$), the local computation delay is calculated as:

$$t_n^l(\tau) = \frac{C_n(\tau)}{F_n^l}, \tag{18}$$

where F_n^l denotes the computing capability of GU_n.

UAV computing: If the task is offloaded to UAV_m (i.e., $x_{n,m}(\tau) = 1$), the delay comprises transmission delay and computation delay on UAV_m. The transmission delay is expressed as:

$$t_{n,m}^{tr}(\tau) = \frac{I_n(\tau)}{R_{n,m}^{G2A}(\tau)}. \tag{19}$$

Let $F_{m,n}(\tau)$ denote the computational resources allocated by UAV_m to GU_n, the execution delay is given by:

$$t_{n,m}^{c}(\tau) = \frac{C_n(\tau)}{F_{m,n}(\tau)}. \tag{20}$$

Total delay: Therefore, if UAV_m has pre-cached the service required for GU_n ($s_{m,g_n}(\tau)=1$), the total processing delay $t_n^{sum}(\tau)$ is:

$$t_n^{\text{sum}}(\tau)=\left(1-\sum_{m=1}^{M}x_{n,m}(\tau)\right)t_n^{l}(\tau)+\sum_{m=1}^{M}x_{n,m}(\tau)\left(t_{n,m}^{c}(\tau)+t_{n,m}^{tr}(\tau)\right). \tag{21}$$

3.5. Energy model

The energy consumption of UAV_m consists of two main components: computation energy for processing tasks and flight energy for mobility.

Computation energy: The energy consumed by UAV_m is given by [42]:

$$E_{n,m}^{c}(\tau)=\lambda_m(F_{m,n}(\tau))^3 t_{m,n}^{c}(\tau), \tag{22}$$

where λ_m is the effective capacitance coefficient of UAV_m. Consequently, the energy consumption of UAV_m for processing tasks in time slot τ can be expressed as:

$$E_m^{c}(\tau)=\sum_{n=1}^{N}x_{n,m}(\tau)E_{n,m}^{c}(\tau). \tag{23}$$

Flight energy: The flight energy $E_m^{\text{f}}(\tau)$ of a rotary-wing UAV depends on its speed v_m, expressed as [42]:

$$E_m^{f}(\tau)=\frac{w_m}{2}\left(\frac{\left|\left|\mathbf{w}_m^{U}(\tau+1)-\mathbf{w}_m^{U}(\tau)\right|\right|}{\Delta\tau}\right)^2\Delta\tau, \tag{24}$$

where w_m denotes the mass of UAV_m.

Energy harvesting: To mitigate limited onboard energy, UAVs are equipped with EH modules, which enable UAVs to capture energy from the wireless environment and store in the energy queue. Let η denote EH efficiency, p_o is the transmission power of energy source, and $G_{m,w}(\tau)$ is the wireless channel gain. Then, the amount of energy harvested by UAV_m in time slot τ is modeled as [43]:

$$A_m(\tau)=\eta p_o G_{m,w}(\tau)\Delta\tau. \tag{25}$$

UAV energy queue: Let $\xi_m^{\max}$ be the maximum battery capacity of UAV_m, and $\xi_m(\tau)$ denotes the energy queue state in time slot τ, which evolves according to:

$$\xi_m(\tau+1)=min\{\xi_m(\tau)+A_m(\tau)-E_m(\tau),\xi_m^{max}\}, \tag{26}$$

where $E_m(\tau)=E_m^{c}(\tau)+E_m^{f}(\tau)$ denote the total energy consumed by UAV_m in time slot τ. To ensure energy queue stability, the following condition must be satisfied:

$$\lim_{\Delta\tau\to T}\frac{\mathbb{E}[|\xi_m(\tau)|]}{\Delta\tau}=0,\forall m. \tag{27}$$

Eq. (27) serves as a long-term energy sustainability criterion, ensuring that the UAV energy queue remains bounded and that the harvested energy is sufficient to sustain long-term operation without persistent energy deficit.

3.6. Problem formulation

Define the set of all decision variables in time slot τ as $\boldsymbol{\theta}(\tau)\triangleq\{x_{n,m}(\tau),F_{m,n}(\tau),\mathbf{w}_m^{U}(\tau),S(\tau)\}$ then the task long-term completion delay can be minimized by jointly optimizing resource allocation, trajectory planning, service caching, and task offloading. The problem is formulated as:

$$\begin{aligned}
\text{P1}:&\min_{\boldsymbol{\theta}(\tau)}\left[\lim_{T\to+\infty}\frac{1}{T}\sum_{\tau=1}^{T}\sum_{n=1}^{N}t_n^{\text{sum}}(\tau)\right]\\
\text{s.t.}\quad &C1:\left\|\mathbf{w}_m^{U}(\tau+1)-\mathbf{w}_m^{U}(\tau)\right\|\le v_{max}\Delta\tau,\forall m,\\
&C2:\left\|\mathbf{w}_m^{U}(\tau)-\mathbf{w}_{m'}^{U}(\tau)\right\|\ge DS_{\min},\forall m\ne m',\\
&C3:\sum_{m=1}^{M}x_{n,m}(\tau)\le 1,\forall n,\\
&C4:\sum_{n=1}^{N}x_{n,m}(\tau)\le N_m^{max},\forall m,\\
&C5:\sum_{j=1}^{J}s_{m,g_j}(\tau)\phi_{g_j}\le\Phi_m^{max},\forall m,\\
&C6:\sum_{n=1}^{N}x_{n,m}(\tau)F_{m,n}(\tau)\le F_m^{\max},\forall m,\\
&C7:x_{n,m}(\tau)\le s_{m,g_n}(\tau),\forall m,n,\\
&C8:\lim_{\Delta\tau\to T}\frac{\mathbb{E}[|\xi_m(\tau)|]}{\Delta\tau}=0,\forall m,\\
&C9:x_{n,m}(\tau)\in\{0,1\},\forall m,n,\\
&C10:s_{m,j}(\tau)\in\{0,1\},\forall m,j,\\
&C11:F_{m,n}(\tau)>0,\forall m,n.
\end{aligned} \tag{28}$$

P1 is a MINLP problem, which is challenging due to several factors. First, the decision variables are heterogeneous, involving binary variables ($x_{n,m}(\tau)$ and $s_{m,g_j}(\tau)$) and continuous variables ($F_{m,n}(\tau)$ and $w_m^{U}(\tau)$). These variables are highly coupled, e.g., the offloading decision (C7, C9) depends on the caching strategy (C5, C10), which in turn influences the resource allocation (C6, C11) and is constrained by the energy queue (C8) affected by UAV mobility (C1, C2). Furthermore, the time-varying channel conditions, UAV positions, and stochastic user demands make this a dynamic optimization problem, rendering traditional static methods ineffective. To tackle these complexities, we decompose P1 into three subproblems, as illustrated in Fig. 2: Service caching subproblem is addressed using CCMAB algorithm to make dynamic caching decisions. Task offloading subproblem is modeled as a PG to design a distributed algorithm for determining offloading strategies. Trajectory planning and resource allocation is solved using MADDPG algorithm to optimize continuous control variables in a collaborative multi-agent setting.

4. CCMAB-Based service caching

Due to limited storage capacity, UAVs can cache only a subset of services. Service caching strategies are formulated using contextual information of associated GUs, under the assumption that GUs with similar contextual information have similar caching demands [44]. This enables UAVs to learn caching preferences online, thereby improving the service cache hit rate.

4.1. Service caching optimization problem

When UAV_m caches the services required by GU_n, it can execute the tasks and obtain corresponding benefits. The reward is defined as the reduction in task completion delay. The smaller execution delay relative to the maximum tolerable latency $T_n(\tau)$, the higher the reward obtained defined as:

$$t_n^{j}(\tau)=\begin{cases}T_n(\tau)-t_{n,m}(\tau),x_{n,m}(\tau)=1 \quad and \quad g_n(\tau)=g_j,\\ 0,otherwise,\end{cases} \tag{29}$$

Given UAV trajectories and offloading decisions, minimizing the total delay is equivalent to maximizing the total reward. The long-term reward maximization problem can be formulated as:

$$\begin{aligned}
&\text{P2}:\max_{S_m(\tau)}\lim_{T\to+\infty}\frac{1}{T}\sum_{\tau=1}^{T}\sum_{m=1}^{M}\sum_{n=1}^{N}t_n^{j}(\tau)\\
&\text{s.t.}\quad C5,C7,C10.
\end{aligned}$$

4.2. Contextual information defination

At the beginning of each time slot, UAV_m first observes the contextual information of its associated GUs represented by a D-dimensional scenario space $K=[0,1]^D$. The contextual information of GUs served by

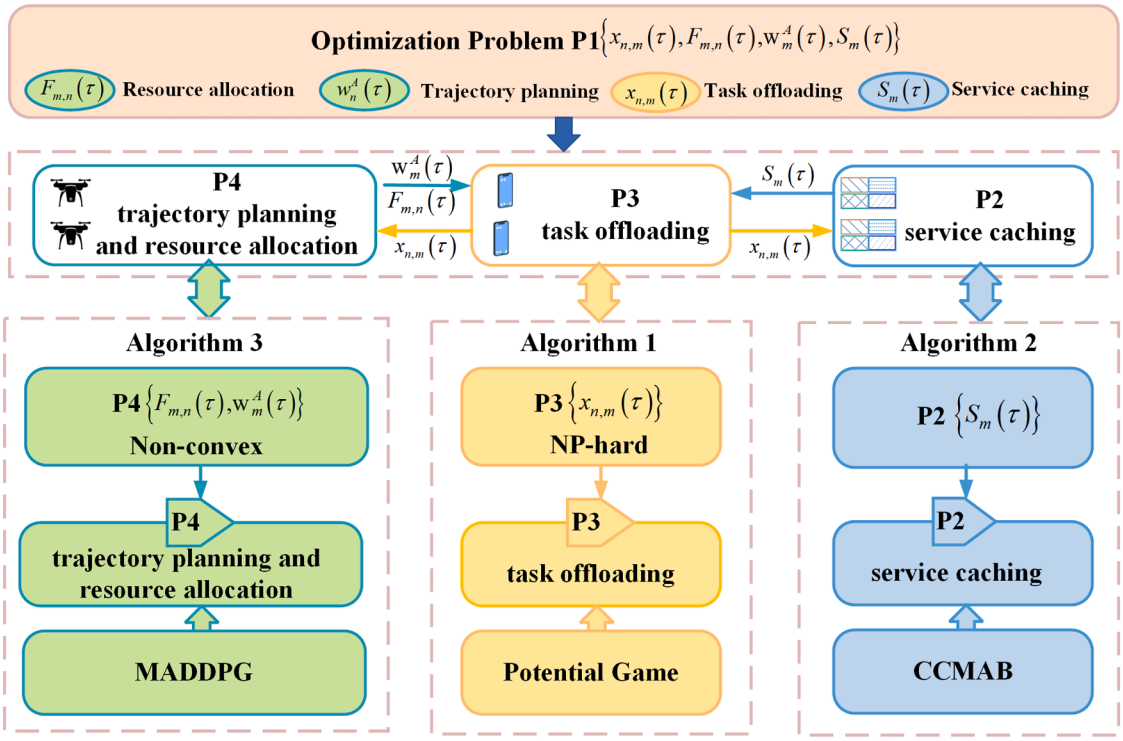


Fig. 2. Flowchart of the CAPG-MADDPG algorithm.

UAV$_m$ forms the set $k_m(\tau) = \{k_{m,n}(\tau), \forall n \in \mathcal{N}\}$, where $k_{m,n}(\tau)$ denotes the contextual information of GU$_n$ associated with UAV$_m$. During initialization, the scenario space K is evenly divided into $(i_T)^D$ cubic subspaces, each with an edge length of i_T per dimension, as shown in Fig. 3. Let $q \in Q_T$ denote the contextual information in each set, where Q_T represents the overall scenario space defined by the partitioning parameters. UAV$_m$ records the relevant information of each partition. The contextual information $k_{m,n}(\tau)$ of each associated GU$_n$ belongs to a corresponding subspace $q_{m,n}(\tau)$, satisfying $k_{m,n}(\tau) \in q_{m,n}(\tau)$. Then, the set of partitioned scenario information for all GUs associated with UAV$_m$ is $q_m(\tau) = \{q_{m,n}(\tau), \forall n \in \mathcal{N}\}$. The execution delay $t_n^j(k_{m,n}(\tau))$ when the task is offloaded to UAV$_m$ is represented as:

$$t_n^j(k_{m,n}(\tau)) = \begin{cases} T_n(\tau) - t_{n,m}(k_{m,n}(\tau)), x_{n,m}(\tau) = 1 \quad and \quad g_n(\tau) = g_j, \\ 0, otherwise. \end{cases} \tag{30}$$

Based on this partitioned scenario information, UAV$_m$ performs service caching to maximize the benefit.

Fig. 3 illustrates a two-dimensional scenario space ($D = 2$). During initialization, the scenario space $K = [0,1]^2$ is evenly divided into $(i_T)^2 = 16$ subspaces with edge length $i_T = 0.25$, forming a 4×4 grid. The figure shows how UAV$_m$ maps the contextual information of its associated GUs to the corresponding subspaces. Specifically, GU$_1$ with $k_{m,1}(\tau) = (0.12, 0.28)$, located in the lower-left region, belongs to subspace $q_{m,1}(\tau) = [0.00, 0.25] \times [0.25, 0.50]$, while GU$_2$ with $k_{m,2}(\tau) = (0.78, 0.73)$, in the upper-right region, belongs to $q_{m,2}(\tau) = [0.75, 1.00] \times [0.50, 0.75]$. This mapping satisfies $k_{m,n}(\tau) \in q_{m,n}(\tau)$. Accordingly, the set of partitioned contextual information for all GUs associated with UAV$_m$ is $q_m(\tau) = \{q_{m,n}(\tau), \forall n \in \mathcal{N}\}$. Based on this discretized scenario space, the CC-MAB algorithm enables UAV$_m$ to learn context-specific caching strategies that adapt to regional service demand characteristics.

Therefore, the optimization problem P2 can be reformulated as:

$$\text{P2.1}: \max_{S_m(\tau)} \lim_{T \to +\infty} \frac{1}{T} \sum_{\tau=1}^{T} \sum_{m=1}^{M} \sum_{k_{m,n}(\tau) \in k_n} \sum_{j=1}^{J} t_n^j(k_{m,n}(\tau))$$

$$\text{s.t.} \quad C5, C7, C10.$$

4.3. CCMAB-Based service caching algorithm

Service caching decisions are updated periodically over a period consisting of T_c time slots. Within each period, the set of cached services at each UAV remains unchanged to ensure operational stability, while contextual observations and execution outcomes are continuously aggregated to refine the estimated rewards. The CCMAB algorithm operates in two phases: exploration and exploitation. In the exploration phase, the algorithm caches services that have not yet been explored or are insufficiently explored for given contexts. After task execution, the algorithm updates the estimated reward based on the actual outcome, thereby refining future caching decisions. In the exploitation phase, UAVs cache the service with the highest estimated reward for the observed context.

(1) *Determining operation phase:* A monotonic increasing control function $F(\tau) = \tau^{\frac{2\zeta}{3\zeta+F}} \log \tau$ is used to ensure sufficient exploration of different caching strategies during the exploration phase [45,46]. Let $\kappa_{q,j}^m(\tau)$ represent the number of times UAV$_m$ has selected service g_j under scenario q by the end of time slot τ. If $\kappa_{q,j}^m(\tau) < F(\tau)$, UAV$_m$ has not fully explored scenario q and service g_j. We use $O_m^{un}(\tau)$ to denote the set of unexplored services by UAV$_m$, defined as $O_m^{un}(\tau) = \{g_j, \forall j \in O_m^{un,num}(\tau)\}$, where $O_m^{un,num}(\tau)$ represents the indices of these insufficiently explored services. The formula for $O_m^{un,num}(\tau)$ is given by:

$$\begin{aligned} O_m^{un,num}(\tau) &= \bigcup_{n \in \mathcal{N}_m} O_{m,q_{n,m}}^{un,num}(\tau), \\ &= \bigcup_{n \in \mathcal{N}_m} \left\{ j \in \mathcal{J} : \kappa_{q,j}^m(\tau) < F(\tau), q_{m,n} \in q \right\}. \end{aligned} \tag{31}$$

(2) *Exploration and exploitation phase:* If $O_m^{un}(\tau)$ is non-empty, UAV$_m$ enters the exploration phase and selects a caching strategy. If $|O_m^{un}(\tau)| \geq \Phi_m^{\max}$, UAV$_m$ randomly caches $S_m^{\max}$ services from $O_m^{un}(\tau)$. Otherwise, UAV$_m$ first caches all services in $O_m^{un}(\tau)$ and selects $SZ = \Phi_m^{\max} - |O_m^{un}(\tau)|$ services with the highest estimated benefits, that is:

$$g_i = \arg \max_{j \in \mathcal{J} \setminus (O_n^{\text{un}}(\tau) \bigcup \bigcup_{s=1}^{i-1} g_s)} t_{q,j}^{m,sum}(\tau), \quad i = 1, 2, \ldots, SZ. \tag{32}$$

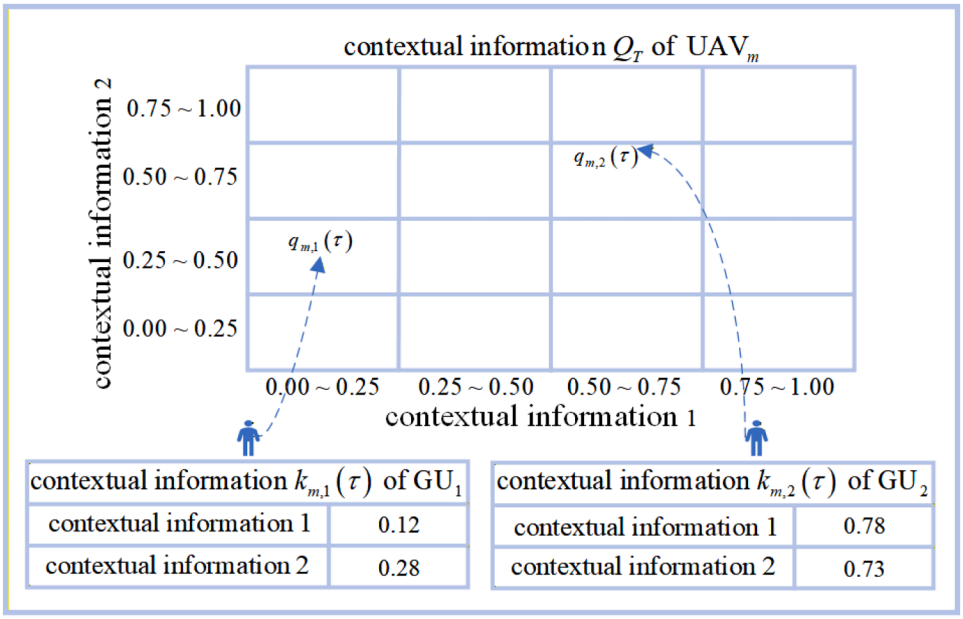


Fig. 3. Example of contextual informantion of UAV$_m$ with GU$_1$ and GU$_2$.

If $O_m^{un}(\tau)$ is empty, UAV$_m$ enters the exploitation phase and selects $G_m^{\max}$ services with the highest estimated benefits, which can be expressed as:

$$g_i = \arg \max_{j \in \mathcal{J} \setminus \bigcup_{s=1}^{i-1} g_s} t_{q,j}^{m,\text{sum}}(\tau), \tag{33}$$

where $t_{q,j}^{m,sum}(\tau)$ denotes the total estimated benefit of UAV$_m$ when the scenario information is q and service g_j is cached, defined as:

$$t_{q,j}^{m,sum}(\tau) = \sum_{k_{m,n}(\tau) \in k_m(\tau)} t_{q,j}^{m}(\tau), \forall m. \tag{34}$$

Let $t_{q,j}^{m}(\tau)$ represent the estimated benefit of UAV$_m$ under scenario information q with service g_j, then the average benefit is updated by:

$$t_{q,j}^{m}(\tau) = \frac{t_{q,j}^{m}(\tau)\kappa_{q,j}^{m,exe}(\tau) + t_j(k_{m,n}(\tau))}{\kappa_{q,j}^{m,exe}(\tau) + 1}, \tag{35}$$

where $\kappa_{q,j}^{m,exe}(\tau)$ denotes the execution count of UAV$_m$. The detailed process of implementing CCMAB for service caching is illustrated in Algorithm 1.

5. PG-Based distributed task offloading

If multiple GUs attempt to offload tasks over the same channel, severe interference may arise, leading to a reduced data transmission rate. If the rate falls below a threshold, tasks will be executed locally. Conversely, under favorable channel conditions, tasks can be offloaded to associated UAV for processing. This analysis motivates the definition of beneficial computation offloading.

Definition 5.1 (Beneficial computation offloading)**.** An offloading decision is considered beneficial if the task completion delay is less than that of local computing, that is:

$$(t_n^{tr}(\tau) + t_{m,n}^{c}(\tau))x_{n,m} < (1 - x_{n,m})t_n^{c}(\tau).$$

Beneficial computation offloading is of significant importance for both GUs and UAVs. It reduces delay and energy consumption for GUs while improving resource utilization and revenue for UAVs. Thus, GUs offload tasks only when it is beneficial. Based on this, the problem of maximizing beneficial computation offloading is formulated as:

$$\text{P3}: \max_{\boldsymbol{x}} \sum_{n=1}^{N}\sum_{m=1}^{M} x_{n,m}(\tau),$$
$$\text{s.t.} \quad C3, C4, C7, C9.$$

Problem P3 is a knapsack-like problem, which is NP-hard [47]. To address this challenge, we propose a distributed algorithm based on PG to determine offloading decisions efficiently.

5.1. PG-Based Multi-GU offloading

For simplicity, the time slot index is omitted in the subsequent discussion. The offloading decision of GU$_n$ is defined as $\boldsymbol{x}_n = (x_{n,1}, x_{n,2}, \ldots, x_{n,M})$, and $\boldsymbol{x}_{-n} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{n-1}, \boldsymbol{x}_{n+1}, \ldots, \boldsymbol{x}_N)$ denotes the decisions of all GUs except GU$_n$. The joint offloading strategy is $\boldsymbol{X} = (\boldsymbol{x}_n, \boldsymbol{x}_{-n})$.

GUs make decisions independently to maximize their individual benefits. The Nash equilibrium ensures system stability when each GU acts in a self-interested manner. If a GU chooses to offload at Nash equilibrium, the resulting latency must be no greater than that of local computing, indicating that GU benefits from offloading. Conversely, if a GU offloads a task at Nash equilibrium without achieve a benefit, it could reduce latency by switching to local computing, which violates Nash equilibrium condition. Therefore, the following corollary holds:

Corollary 5.1. At Nash equilibrium $\mathbf{X}^* = (\mathbf{x}_1^*, \ldots, \mathbf{x}_N^*)$, if GU$_n$ offloads a task to UAV$_m$ (i.e., $x_{n,m} = 1$), then the offloading must be beneficial, satisfying: $t_n^c(\tau) > t_n^{tr}(\tau) + t_{m,n}^c(\tau)$.

By optimizing the offloading decision, the task completion delay for GU$_n$ can be minimized:

$$\min_{\mathbf{x}_n} t_n^{sum}(\mathbf{x}_n, \mathbf{x}_{-n}), \tag{36}$$

where the total cost function $t_n^{sum}(\mathbf{x}_n, \mathbf{x}_{-n})$ is defined as:

$$t_n^{sum}(\mathbf{x}_n, \mathbf{x}_{-n}) = \begin{cases} t_n^c, & x_{n,m} = 0, \\ t_{n,m}^c(\tau) + t_{n,m}^{tr}(\tau), & x_{n,m} = 1. \end{cases} \tag{37}$$

The problem is modeled as a strategy game: $GM = \left(N, \{\mathbf{x}_n\}_{n \in \mathcal{N}}, \{t_n^{sum}\}\right)$. There exists a potential function $\phi(\mathbf{X})$ such

Algorithm 1 CCMAB-based service caching algorithm.

Input: N_m, $k_m(\tau)$, $\Phi_m^{\max}$, $\mathcal{G}$, T_c;
Output: $S(\tau) = \{S_m(\tau) \mid m \in \mathcal{M}\}$;
Initialization: $\kappa_{q,j}^m(\tau)$, $t_{q,j}^m(\tau)$;
for $\tau = 1$ to T **do**
  for $m \in \mathcal{M}$ **do**
    Collect contextual information $k_m(\tau)$ for all GUs;
    Map contextual information to partition $q \in Q_T$;
    if $\tau \bmod T_c = 0$ **and** $O_m^{un}(\tau) \neq \emptyset$ **then**
      Exploration phase
      if $|O_m^{un}(\tau)| \geq \Phi_m^{\max}$ **then**
        UAV_m randomly caches $\Phi_m^{\max}$ services from $O_m^{un}(\tau)$;
        Update caching strategy $S_m(\tau)$;
      else
        UAV_m caches all services in $O_m^{un}(\tau)$;
        Cache SZ services with highest $t_{q,j}^m(\tau)$ from explored services;
        Update caching strategy $S_m(\tau)$;
      end if
    else
      Exploitation phase
      UAV_m caches $\Phi_m^{\max}$ services with highest estimated rewards;
      Update caching strategy $S_m(\tau)$;
      Update $\kappa_{q,j}^m(\tau) \leftarrow \kappa_{q,j}^m(\tau) + 1$;
    end if
    if $\tau \bmod T_c \neq 0$ **then**
      Keep caching strategy unchanged $S_m(\tau) = S_m(\tau - 1)$;
    end if
    for $n \in N_m$ **do**
      Obtain service request $g_n(\tau)$ of GU_n, $j \leftarrow g_n(\tau)$;
      if $s_{m,g_j}(\tau) = 1$ **then**
        Perform task offloading;
        $\kappa_{q,j}^{m,exe}(\tau) \leftarrow \kappa_{q,j}^{m,exe}(\tau) + 1$;
        Update estimated reward $t_{q,j}^m(\tau)$;
      else
        Execute locally;
      end if
    end for
  end for
end for

that any unilateral strategy change by a GU that reduces its completion delay also decreases the potential function [44], that is:

$$t_n^{\text{sum}}(\mathbf{x}_n', \mathbf{x}_{-n}) \leq t_n^{\text{sum}}(\mathbf{x}_n, \mathbf{x}_{-n}) \quad \Rightarrow \quad \phi(\mathbf{x}_n', \mathbf{x}_{-n}) \leq \phi(\mathbf{x}_n, \mathbf{x}_{-n}). \tag{38}$$

Therefore, demonstrating the existence of a Nash equilibrium is equivalent to establishing the existence of a global potential function. The following theorem characterizes potential function for the multi-GU offloading game.

Theorem 5.1. The multi-GU task offloading game constitutes a PG with $\phi(\mathbf{X})$ as potential function, defined as:

$$\phi(\boldsymbol{X}) = \sum_{n \in \mathcal{N}} t_n^{sum}(\boldsymbol{X}). \tag{39}$$

Proof. Suppose GU_n updates its offloading strategy from $\mathbf{x}_n$ to $\mathbf{x}_n'$, resulting in a reduction in cost function, that is:

$$\phi(\mathbf{x}_n, \mathbf{x}_{-n}) - \phi(\mathbf{x}_n', \mathbf{x}_{-n}) = \sum_{n \in \mathcal{N}} t_n^{sum}(\mathbf{x}_n, \mathbf{x}_{-n}) - \sum_{n \in \mathcal{N}} t_n^{sum}(\mathbf{x}_n', \mathbf{x}_{-n}) > 0. \tag{40}$$

GUs strategy updates can be categorized into three cases:

Case I: When GU transitions from local computing to offloading, the change in potential function is:

$$\begin{aligned}\phi(\mathbf{x}_n, \mathbf{x}_{-n}) - \phi(\mathbf{x}_n', \mathbf{x}_{-n}) &= t_n^c - \left(t_{n,m}^{tr} + t_{n,m}^c\right), \\ &= \frac{C_n}{F_1} - \left(\frac{I_n}{R_{G2A}^{n,m}} + \frac{C_n}{F_{m,n}}\right) > 0.\end{aligned} \tag{41}$$

Case II: When GU switches from offloading to local computing, the resulting change in the potential function is:

$$\begin{aligned}\phi(\mathbf{x}_n, \mathbf{x}_{-n}) - \phi(\mathbf{x}_n', \mathbf{x}_{-n}) &= \left(t_{n,m}^{tr} + t_{n,m}^c\right) - t_n^c, \\ &= \left(\frac{I_n}{R_{G2A}^{n,m}} + \frac{C_n}{F_{m,n}}\right) - \frac{C_n}{F_n^l} > 0.\end{aligned} \tag{42}$$

Case III: When a GU switches offloading strategy from UAV_m to UAV_m', the corresponding change is:

$$\begin{aligned}\phi(\mathbf{x}_n, \mathbf{x}_{-n}) - \phi(\mathbf{x}_n', \mathbf{x}_{-n}) &= \left(\frac{I_n}{R_{G2A}^{n,m}} + \frac{C_n}{F_{m,n}}\right) - \left(\frac{I_n}{R_{G2A}^{n,m'}} + \frac{C_n}{F_{m',n}}\right) \\ &= I_n\left(\frac{1}{R_{G2A}^{n,m}} - \frac{1}{R_{G2A}^{n,m'}}\right) + C_n\left(\frac{1}{F_{m,n}} - \frac{1}{F_{m',n}}\right) > 0.\end{aligned} \tag{43}$$

□

As discussed above, the change in potential function aligns with the change in delay for GU_n. Therefore, the multi-GU offloading game constitutes a PG.

5.2. Distributed offloading algorithm

The distributed task offloading algorithm based on PG is summarized in Algorithm 2. In each iteration, GUs evaluate all available UAVs and select the one that minimizes offloading delay. If a UAV reaches its maximum service capacity, GU may replace an existing GU with the highest delay on that UAV to further reduce delay. Each GU updates its offloading decision only if the change guarantees a strict reduction in task completion delay. This update rule prevents convergence to equilibria that are locally stable but inefficient. The algorithm converges when the change in the potential function $\Delta\Phi(\mathbf{X})$ falls below the threshold ϵ.

Definition 5.2 (Nash equilibrium)**.** A strategy $\boldsymbol{X}^* = (\boldsymbol{x}_1^*, \ldots, \boldsymbol{x}_N^*)$ is a Nash equilibrium if no GU can unilaterally change offloading strategy to reduce its completion delay [48]. Formally, for each GU_n, the following condition holds:

$$t_n^{sum}(\boldsymbol{x}_n^*, \boldsymbol{x}_{-n}^*) \leq t_n^{sum}(\boldsymbol{x}_n, \boldsymbol{x}_{-n}^*), \forall n. \tag{44}$$

6. MADDPG-Based trajectory and resource optimization

After the offloading strategy is determined via PG, the continuous optimization of UAV trajectory and resource allocation is addressed using MADDPG. Compared with traditional Q-learning or policy gradient algorithms, MADDPG is better suited for multi-agent environment and maintains the advantage of handling continuous action spaces.

Given offloading decisions and caching strategy, problem P1 can be simplified as:

$$\begin{aligned}&\text{P3}: \min_{\mathbf{w}_m^U(\tau), F_{m,n}(\tau)} \sum_{m=1}^{M} \sum_{n=1}^{N} x_{n,m}(\tau)\left(t_{n,m}^c(\tau) + t_{n,m}^{tr}(\tau)\right), \\ &\text{s.t.} \quad C1, C2, C6, C8, C11\end{aligned} \tag{45}$$

6.1. MDP modeling

The position of a UAV in the next time slot depends only on its current position and actions (flight azimuth θ_m and velocity υ_m), and is independent of its historical trajectory. This satisfies the Markov property, allowing the problem to be modeled as a MDP, defined as follows:

Local state: Each UAV observes a local state within its vicinity, including its own position $\mathbf{w}_m^U(\tau)$, offloading decision $x_{n,m}(\tau)$, and the position of associated GU_n, $\mathbf{w}_n^A(\tau)$. The local state of UAV_m at time slot τ is denoted by $s_m(\tau) = \{\mathbf{w}_m^U(\tau), \mathbf{w}_n^A(\tau), x_{n,m}\}$, and the joint states of all UAVs is represented by $\mathbf{s}(\tau) = \{s_1(\tau), \ldots, s_M(\tau)\}$.

Algorithm 2 PG-based distributed offloading algorithm.

Input: $\mathcal{N}$, $\mathcal{M}$, $F_m^{\max}$, $\mathbf{W}^A$, $\mathbf{W}^U$, $N_m^{\max}$;
Output: $\mathbf{X} = [x_{n,m}]$;
Initialize $\mathbf{X} = \mathbf{0}$, $\Phi(\mathbf{X})$, $N_m^u = 0$;
for $n \in \mathcal{N}$ **do**
  Set minimum delay $t_n^{\min} = t_n^l$, $m^* = -1$;
  for $m \in \mathcal{M}$ **do**
    if $d_{n,m} \le D_{G2A}$ **and** $s_{m,g_j} = 1$ **then**
      Compute offloading delay $t_{n,m}$ based on $F_{m,n}$;
      if $N_m^u < N_m^{\max}$ **and** $t_{n,m} < t_n^{\min}$ **then**
        $t_n^{\min} = t_{n,m}$;
        $m^* = m$;
      else if $N_m^u \ge N_m^{\max}$ **then**
        Find $\mathrm{GU}_{n'}$ on UAV_m with the largest offloading delay;
        if $t_{n,m} < t_{n',m}$ **then**
          $m^* = m$, replace $\mathrm{GU}_{n'}$ with GU_n;
        end if
      end if
    end if
    if $m^* = -1$ **then**
      $x_{n,m} = 0$;
    else
      $x_{n,m^*} = 1$;
    end if
  end for
end for
Update potential function $\Phi(\mathbf{X})$;

Action: The action of UAV_m at time slot τ comprises three components: the flight angle $\theta_m(\tau)$, the velocity $v_m(\tau)$, and the allocated resource $F_{m,n}(\tau)$, defined as $a_m(\tau) = \{\theta_m(\tau), v_m(\tau), F_{m,n}(\tau)\}$. The joint action of all UAVs is $a(\tau) = \{a_1(\tau), \ldots, a_M(\tau)\}$

Reward function: The reward $r_m(\tau)$ evaluates the effectiveness of UAV_m after executing action $a_m(\tau)$. The set of rewards for all UAVs is $r(\tau) = \{r_1(\tau), \ldots, r_M(\tau)\}$. To satisfy constraint $C2$ and ensure operation within the service area, a penalty term PM is imposed for violations. The reward is computed as:

$$r_m(\tau) = \sum_{n=1}^{N} -x_{n,m}(\tau)\left(t_{n,m}^c(\tau) + t_{n,m}^{tr}(\tau)\right) - PM \tag{46}$$

6.2. MADDPG algorithm design

MADDPG algorithm operates within the actor-critic framework as shown in Fig. 4. Each agent consists of an online actor network $\rho_m(s_m(\tau) \mid \mu^\rho)$, an online critic network $Q_m(s(\tau), a(\tau) \mid \mu^Q)$, and the corresponding target networks $\rho'_m(s_m(\tau) \mid \mu^{\rho'})$ and $Q_m(s(\tau), a(\tau) \mid \mu^{Q'})$. The actor network observes the current state and outputs an appropriate action, while the critic network evaluates the quality of action. To reduce sample correlation and improve learning stability and efficiency, an Experience Replay Buffer (ERB) is employed. The buffer stores interaction experiences, represented as $\{s(\tau), a(\tau), r(\tau), s(\tau+1)\}$.

During training, each agent observes its local state $s_m(\tau)$, and the actor network $\rho_m(s_m(\tau) \mid \mu^\rho)$ generates an action based on $s_m(\tau)$. To improve exploration and avoid convergence to local optima, Gaussian noise σ is added, expressed as:

$$a_m(\tau) = \rho_m(s_m(\tau) \mid \mu^\rho) + \sigma, \tag{47}$$

where μ^ρ denotes the parameters of actor network.

The critic network $Q_m(s(\tau), a(\tau) \mid \mu^Q)$ estimates the action-value function Q_m given the global state $s(\tau)$ and global action $a(\tau)$. The parameters μ^Q are updated by minimizing the Bellman loss:

$$L(\mu^Q) = \mathbb{E}_{s(\tau),a(\tau),r(\tau),s(\tau+1)}\left[\left(Q_m(s(\tau), a(\tau) \mid \mu^Q) - Y_m\right)^2\right], \tag{48}$$

where μ^Q denotes the critic network parameters, and Y_m is the target Q-value, given by:

$$Y_m = r_m(\tau) + \gamma * Q'_m\left(s(\tau+1), a'_m(\tau+1) \mid \mu^{Q'}\right), \tag{49}$$

where γ is the discount factor, Q'_m is the target critic network, and $a'_m(\tau+1)$ denotes the next action predicted by the target actor network $\rho'_m(s(\tau+1) \mid \mu^{\rho'})$, expressed as

$$a'_m(\tau+1) = \rho'_m(s(\tau+1) \mid \mu^{\rho'}). \tag{50}$$

The policy parameters μ^ρ of actor network are updated using policy gradient to maximize the policy objective function $U(\mu^\rho)$:

$$\begin{aligned} U(\mu^\rho) = & \sum_{s_m(\tau) \in s(\tau)} b^\rho(s_m(\tau)) \\ & \times \sum_{a_m(\tau) \in a(\tau)} \rho_m(a_m(\tau), s_m(\tau) \mid \mu^\rho) Q_m(s(\tau), a(\tau) \mid \mu^Q). \end{aligned} \tag{51}$$

where $b^\rho(s_m(\tau))$ denotes the stationary distribution induced by policy ρ.

The update rule for the actor network policy parameters μ^ρ is:

$$\mu^\rho = \mu^\rho + \iota \nabla_\mu U(\mu^\rho), \tag{52}$$

where ι denotes the learning rate, and $\nabla_\mu U(\mu^\rho)$ is the policy gradient:

$$\nabla_\mu U(\mu^\rho) = \mathbb{E}\left[\nabla_{a_m(\tau)} Q_m(s(\tau), a(\tau) | \mu^Q) \cdot \nabla_\mu \mu_m(s_{\mathrm{m}}(\tau) | \mu^\rho)\right]. \tag{53}$$

The update rule for critic network parameters is given by:

$$Q^\rho = Q^\rho + \iota \nabla Q_m(s(\tau), a(\tau) \mid \mu^Q). \tag{54}$$

To ensure stable training, MADDPG algorithm adopts a soft update strategy to gradually adjust the target network parameters:

$$\begin{cases} \mu^{\rho'} \leftarrow \varepsilon \mu^\rho + (1-\varepsilon)\mu^{\rho'}, \\ Q^{\rho'} \leftarrow \varepsilon Q^\rho + (1-\varepsilon) Q^{\rho'}, \end{cases} \tag{55}$$

where ε is the soft update coefficient with $\varepsilon \ll 1$.

Algorithm 3 outlines the complete training process for optimizing trajectory and resource allocation using MADDPG. Over a total of CL training episodes, agents iteratively observe states, select actions with exploration noise, receive rewards, and update network parameters via mini-batches sampled from the ERB.

Algorithm 3 Trajectory and resource optimization based on MADDPG.

Input: CL, M, $\rho_m(s_m(\tau) \mid \mu^\rho)$, $Q_m(s(\tau), a(\tau) \mid \mu^Q)$;
Initialization: $\mu^{\rho'} \leftarrow \mu^\rho$, $\mu^{Q'} \leftarrow \mu^Q$ and ERB = $\varnothing$;
for $cl = 1$ to CL **do**
  Update offloading decision $\mathbf{X}$ according to Algorithm 2;
  for $\tau = 1$ to T **do**
    Obtain global state $s(\tau)$;
    for each agent $m \in \mathcal{M}$ **do**
      Select action $a_m(\tau) = \rho_m(s_m(\tau) \mid \mu^\rho) + \sigma$;
    end for
    Execute action $a_m(\tau)$ and compute reward $r_m(\tau)$;
    Store $\{s(\tau), a(\tau), r(\tau), s(\tau+1)\}$ into ERB;
    if ERB is full **then**
      Randomly sample a mini-batch for training;
      for each agent $m \in \mathcal{M}$ **do**
        Train online critic network using loss in Eq. (48);
        Update online actor network using Eq. (52);
        Soft-update target actor and critic networks using Eq. (55);
      end for
    end if
  end for
end for

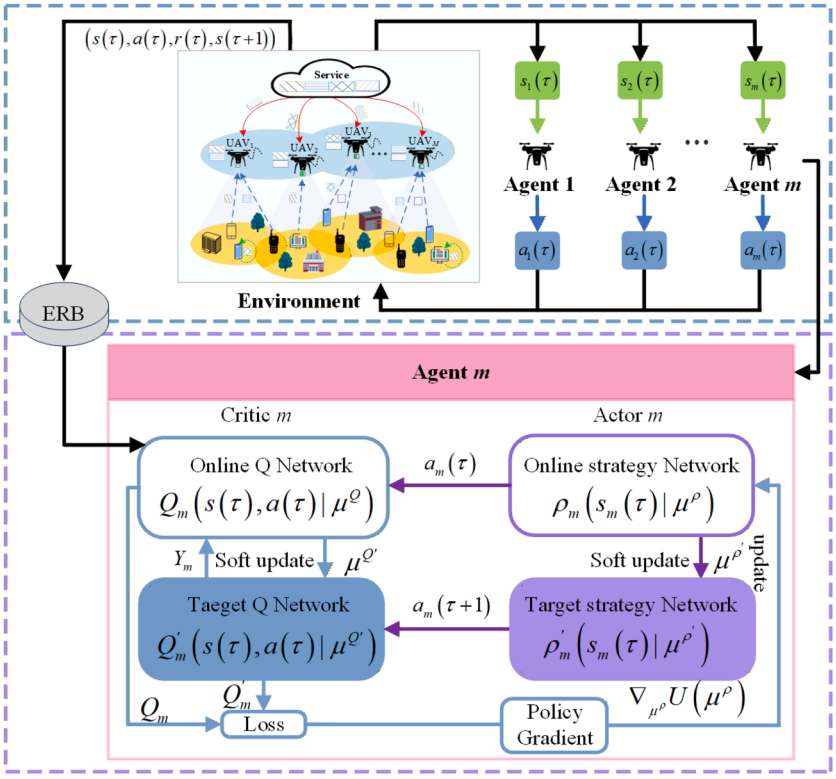


Fig. 4. MADDPG-based trajectory and resource optimization.

Table 2
Simulation Parameters.

Parameter	Symbol	Value
UAV maximum capacity	$F_m^{\max}$	5 GHz
CPU cycles required to process task	$C_n(\tau)$	1000 cycles/bit
Transmission power of GU_n	$p_n^{tr}(\tau)$	6 W
Communication system bandwidth	B	40 MHz
Gaussian noise	σ	-80 dBm
Carrier frequency	f_c	2.5 GHz

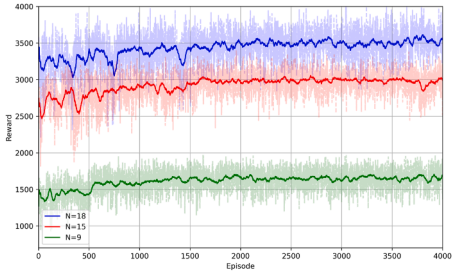


Fig. 5. Convergence performance of CAPG-MADDPG under different network sizes.

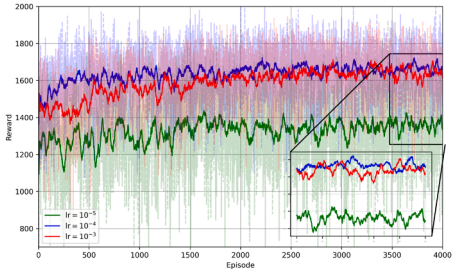


Fig. 6. Convergence performance of CAPG-MADDPG under different learning rates.

7. Simulation results

This section conducts simulation experiments to evaluate the performance of proposed CAPG-MADDPG algorithm in a multi-user, multi-UAV-assisted MEC network. The experiments are implemented on a Python 3.7 platform, running on an AMD R7 processor with 16 GB of RAM and a 3.6 GHz CPU. UAVs are deployed within a square area of 1×1 km, which is normalized to a 1×1 unit square in a two-dimensional coordinate system for visualization. The maximum number of training episodes is set to 5000, with a step size T of 100 per episode. Detailed simulation parameters are provided in Table 2.

7.1. Convergence analysis

This section examines the convergence behaviour of proposed CAPG-MADDPG under different network configurations and hyperparameter settings.

Network size: Fig. 5 illustrates the convergence performance of CAPG-MADDPG under different network sizes, where three UAVs serve 9, 15, and 18 GUs.

With fewer GUs, rapid convergence to a stable reward is observed due to lower interaction complexity. In contrast, larger network sizes introduceThese results demonstrate the robustness of CAPG-MADDPG across different GUs densities. more complex user interactions, leading to slower convergence, although stable convergence is still achieved.

Learning rate (lr): Fig. 6 depicts the convergence behavior of CAPG-MADDPG under learning rates set to 1×10^{-5}, 1×10^{-4}, and 1×10^{-3}. When the learning rate is 1×10^{-3}, convergence is slow with significant oscillations and the lowest steady-state reward. Reducing the learning rate to 1×10^{-4} results in faster convergence, smoother learning trajecto-

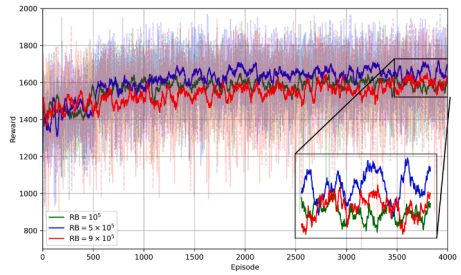


Fig. 7. Convergence performance of CAPG-MADDPG under different replay buffers.

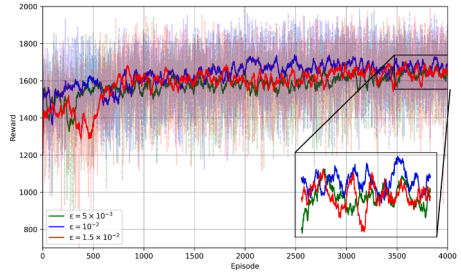


Fig. 8. Convergence performance of CAPG-MADDPG under different soft update coefficients.

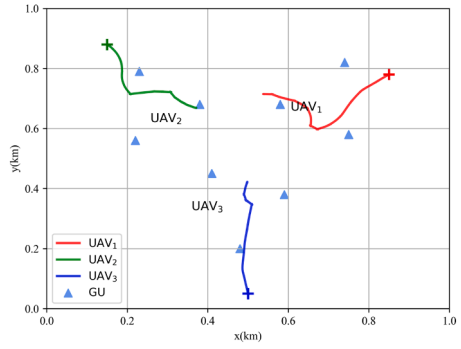


Fig. 9. UAV trajectory under different initial positions (N = 9).

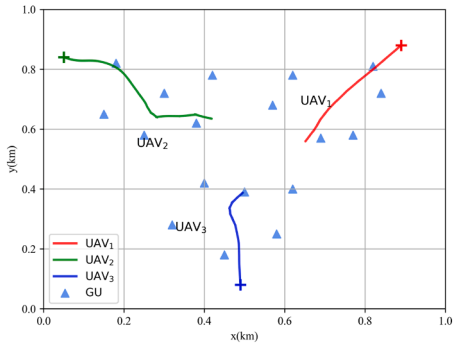


Fig. 10. UAV trajectory under different initial positions (N = 18).

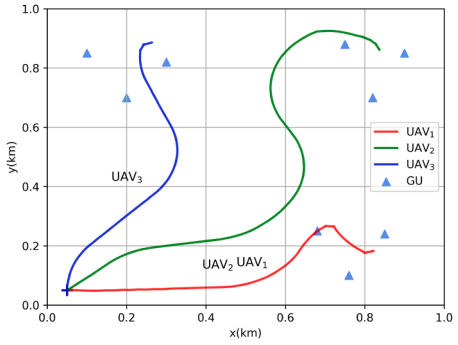


Fig. 11. UAV trajectory under common initial positions (N = 9).

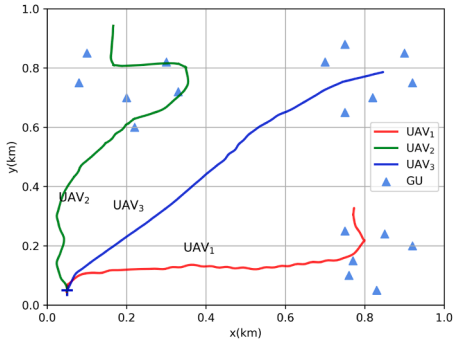


Fig. 12. UAV trajectory under common common positions (N = 18).

ries, and the highest stable reward. A smaller learning rate of 1×10^{-5} remains stable but converges too slowly to eliminate the performance gap within the training horizon. Overall, 1×10^{-4} provides the best trade-off between convergence speed and training stability.

Replay Buffer(RB): The impact of Replay Buffer Capacity (RBC) is shown in Fig. 7, using capacities of 1×10^4, 5×10^4, and 9×10^4. A small buffer leads to faster early convergence but causes fluctuations, while a large buffer enhances stability at the cost of slower convergence and higher memory usage. A medium buffer achieves the most balanced and stable performance.

Soft update coefficient: Fig. 8 shows the convergence performance of CAPG-MADDPG under different soft update coefficients ε of 5×10^{-3}, 10^{-2}, and 1.5×10^{-2}. A smaller ε leads to slower early convergence but improved training stability, while a larger ε accelerates convergence at the cost of increased performance fluctuations. Overall, a moderate ε provides a better balance between convergence speed and stability.

7.2. UAV trajectory analysis

This section evaluates the trajectory planning performance of proposed CAPG-MADDPG algorithm under varying network scales and initial UAV configurations.

Different initial UAV positions: Fig. 9 and Fig. 10 illustrate the UAV trajectory under different initial UAV positions. Triangles denote GUs, and coloured crosses indicate the initial UAV positions.

As shown in Fig. 9, in small-scale networks, the proposed algorithm enables each UAV to adjust its flight direction according to the spatial distribution of GUs around it, enabling it to approach the target service area progressively. Even in large-scale settings with denser user distribution, the algorithm effectively guides UAVs to dynamically adjust their trajectories in order to serve distinct GU clusters, thereby demonstrating adaptability to variations in network scale.

Common initial UAV positions: Fig. 11 and Fig. 12 show the UAV trajectory under common initial UAV positions.

When departing from the same starting point, multiple UAVs overcome the homogenising effect of shared initial conditions through a cooperative division of labour, with each UAV flying towards separate GU clusters. This confirms that the proposed CAPG-MADDPG promotes collaborative decision-making even under identical initial conditions.

As depicted in these figures, the CAPG-MADDPG algorithm exhibits consistent trajectory planning performance regardless of network size or initial configuration. This capability allows UAVs to achieve effective

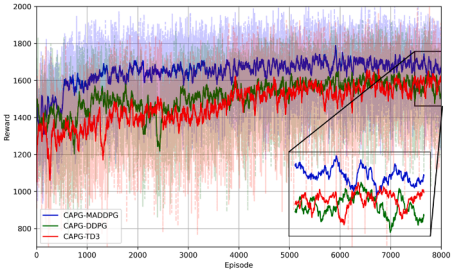


Fig. 13. Convergence performance under different DRL.

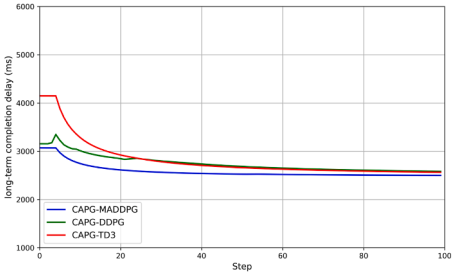


Fig. 14. Convergence comparison for different DRL.

collaboration and labour division in dynamic environments, thereby extending the service coverage range of MEC system.

7.3. System utility analysis

We compare the performance of the proposed CAPG-MADDPG with the following algorithms:

(1) *Context-Aware Potential Game-based DDPG (CAPG-DDPG):* This algorithm adopts DDPG framework to optimize decision-making in continuous action spaces using deterministic policy gradients.
(2) *Context-Aware Potential Game-based TD3 algorithm (CAPG-TD3):* This algorithm employs TD3 framework, an enhanced variant of DDPG to improve training stability and alleviate overestimation bias.
(3) *Context-Aware Random Matching MADDPG (CARM-MADDPG):* This algorithm adopts a random matching strategy to determine offloading decisions for GUs.
(4) *Context-Aware Greedy Matching MADDPG (CAGM-MADDPG):* In this scheme, greedy matching is used to determine offloading decisions based on the current system state.
(5) *Random Caching PG-MADDPG (RCPG-MADDPG):* Services are randomly cached within the MADDPG framework.
(6) *Multi-Armed Bandit Caching PG-MADDPG (MAPG-MADDPG):* This algorithm uses a MAB caching strategy to select optimal caching services for UAVs based on historical reward feedback.

To ensure fair comparisons, we evaluate the different algorithmic components in various experimental settings. The trajectory planning and task matching modules were tested using 9 users, 3 UAVs and 6 service types. By contrast, caching is evaluated in a more challenging scenario involving 12 service types, which highlights the benefits of intelligent caching more effectively.

Fig. 13 shows the rewards of CAPG-DDPG, CAPG-TD3 and CAPG-MADDPG exhibit an increasing trend with the progression of training episodes. However, CAPG-DDPG and CAPG-TD3 converge more slowly and display greater reward volatility than CAPG-MADDPG. By contrast, the CAPG-MADDPG exhibits faster convergence, higher steady-state rewards and enhanced stability in multi-UAV coordination scenarios. This is due to its Critic-Actor architecture and MADDPG's capacity to leverage global information.

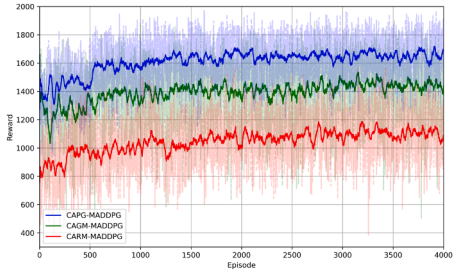


Fig. 15. Convergence comparison for different matching strategies.

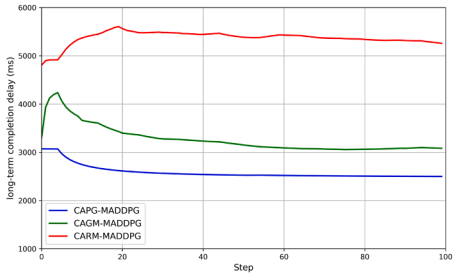


Fig. 16. Delay comparison for different matching strategies.

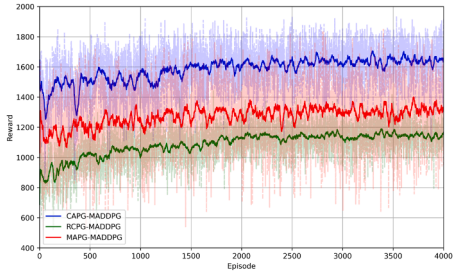


Fig. 17. Convergence comparison for differentcaching strategies.

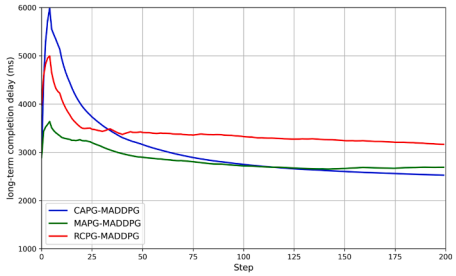


Fig. 18. Delay comparison for different caching strategies.

Fig. 14 shows the long-term task completion delays for different DRL methods. All three approaches exhibit a rapid initial reduction in total delay, followed by gradual convergence. CAPG-MADDPG consistently achieves the lowest long-term delay, outperforming both CAPG-TD3 and CAPG-DDPG.

Furthermore, the performance comparion of matching strategies are shown in Fig. 15 and Fig. 16 how rewards change for different matching strategies during training episodes. The proposed CAPG-MADDPG algorithm, iteratively approximates the Nash equilibrium during the matching phase, resulting in higher and smoother reward. In contrast, CAGM-MADDPG focuses solely on maximising instantaneous gains, and CARM-MADDPG fails to improve cache hit rates, both leading to lower overall rewards due to the lack of global coordination.

Similarly, as depicted in Fig. 16, in terms of delay reduction, CAPG-MADDPG achieves the lowest overall delay rapidly, whereas CAGM-

MADDPG often settles at a local optimum and CARM-MADDPG performs poorly due to its random matching strategy.

Fig. 17 compares the rewards achieved by CAPG-MADDPG, which incorporates a context-aware caching mechanism, achieves higher reward more quickly and with lower volatility compared to GCPG-MADDPG and RCPG-MADDPG. The adaptive caching mechanism in CAPG-MADDPG effectively exploits contextual information to match dynamic user demands. In contrast, RCPG-MADDPG relies on random caching without demand awareness, resulting in the lowest steady-state rewards and significant fluctuations. Fig. 18 shows the long-term task completion delays for the various caching strategies. All three approaches initially experience a rapid reduction in total delay, followed by gradual convergence. Although CAPG-MADDPG initially experiences higher delays, it ultimately achieves the lowest long-term delay, outperforming both MAPG-MADDPG and RCPG-MADDPG. CAPG-MADDPG, GCPG-MADDPG, and RCPG-MADDPG.

In summary, the proposed CAPG-MADDPG algorithm outperforms all other methods in reward, convergence speed, and long-term delay, validating the effectiveness of its integrated PG, and context-aware matching in multi-UAV scenarios.

8. Conclusion

This paper investigates the joint optimization of trajectories, resource allocation, service caching, and task offloading in a multi-UAV-assisted MEC architecture. The original MINLP problem is decomposed into task offloading subproblem, service caching subproblem, trajectory planning and resources allocation subproblem, and a CAPG-MADDPG algorithm is proposed to jointly solve this problem. Owing to its distributed architecture and adaptive learning capability, the proposed CAPG-MADDPG algorithm can be deployed in dynamic practical scenarios that require low-latency edge services. Examples of these scenarios include emergency response, temporary hotspot coverage, and areas with limited communication infrastructure. Simulation results demonstrate that the proposed CAPG-MADDPG algorithm achieves minimal long-term task completion delay while maintaining stability and robustness under dynamic network conditions. However, the current study assumes quasi-static GUs and does not explicitly model user mobility or information freshness requirements. Future work will extend the framework to incorporate user mobility dynamics, delay-sensitive tasks, and age-of-information metrics. It will also develop proactive UAV service strategies, investigate inter-UAV collaborative service provisioning and evaluate performance under more complex operational conditions, thereby further enhancing the practical applicability of the proposed algorithm.

CRediT authorship contribution statement

Zilin Wang: Writing – original draft; **Liming Chen:** Writing – review & editing; **Pengxian Chen:** Visualization; **Yifeng Zheng:** Visualization; **Wenjie Zhang:** Conceptualization.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by National Natural Science Foundation of China (No.62372488), Natural Science Foundation of Fujian Province (No.2025J01366, No.2024J011560).

References

[1] J. Tian, D. Wang, H. Zhang, D. Wu, Service satisfaction-oriented task offloading and UAV scheduling in UAV-enabled MEC networks, IEEE Trans. Wirel. Commun. 22 (12) (2023) 8949–8964. https://doi.org/10.1109/TWC.2023.3267330
[2] Y. Zeng, S. Chen, Y. Ge, Star-RIS-assisted UAV-enabled MEC network: minimizing long-term latency and system stability optimization, Comput. Netw., 2025, 111563. https://doi.org/10.1016/j.comnet.2025.111563
[3] H. Yang, K.-Y. Lam, L. Xiao, Z. Xiong, H. Hu, D. Niyato, H.V. Poor, Lead federated neuromorphic learning for wireless edge artificial intelligence, Nat. Commun. 13 (1) (2022) 4269. https://doi.org/10.1038/s41467-022-32020-w
[4] Y. Chen, K. Li, Y. Wu, J. Huang, L. Zhao, Energy efficient task offloading and resource allocation in air-ground integrated MEC systems: a distributed online approach, IEEE Trans. Mobile Comput. 23 (8) (2024) 8129–8142. https://doi.org/10.1109/TMC.2023.3346431
[5] M. Hui, J. Chen, L. Yang, L. Lv, H. Jiang, N. Al-Dhahir, Uav-assisted mobile edge computing: optimal design of UAV altitude and task offloading, IEEE Trans. Wireless Commun. 23 (10) (2024) 13633–13647. https://doi.org/10.1109/TWC.2024.3403536
[6] H. Yang, S. Liu, L. Xiao, Y. Zhang, Z. Xiong, W. Zhuang, Learning-based reliable and secure transmission for UAV-RIS-assisted communication systems, IEEE Trans. Wireless Commun. 23 (7) (2024) 6954–6967. https://doi.org/10.1109/TWC.2023.3336535
[7] M. Zhao, R. Zhang, Z. He, K. Li, Joint optimization of trajectory, offloading, caching, and migration for UAV-assisted MEC, IEEE Trans. Mobile Comput. 24 (3) (2025) 1981–1998. https://doi.org/10.1109/TMC.2024.3486995
[8] Z. Hu, F. Zeng, Z. Xiao, B. Fu, H. Jiang, H. Chen, Computation efficiency maximization and QoE-provisioning in UAV-enabled MEC communication systems, IEEE Trans. Netw. Sci. Eng. 8 (2) (2021) 1630–1645. https://doi.org/10.1109/TNSE.2021.3068123
[9] Y. Zeng, S. Chen, J. Li, Y. Cui, J. Du, Online optimization in UAV-enabled MEC system: minimizing long-term energy consumption under adapting to heterogeneous demands, IEEE Internet Things J. 11 (19) (2024) 32143–32159. https://doi.org/10.1109/JIOT.2024.3426312
[10] L. Wang, K. Wang, C. Pan, N. Aslam, Joint trajectory and passive beamforming design for intelligent reflecting surface-aided UAV communications: a deep reinforcement learning approach, IEEE Trans. Mobile Comput. 22 (11) (2023) 6543–6553. https://doi.org/10.1109/TMC.2022.3200998
[11] Y. Wang, H. Wang, J. Ding, H. Yu, From latency bottlenecks to seamless edge: AD3PG-powered joint optimization of UAV trajectory and task offloading, Comput. Netw., 2025, 111700. https://doi.org/10.1016/j.comnet.2025.111700
[12] Y. Liu, S. Xie, Y. Zhang, Cooperative offloading and resource management for UAV-enabled mobile edge computing in power IoT system, IEEE Trans. Veh. Technol. 69 (10) (2020) 12229–12239. https://doi.org/10.1109/TVT.2020.3016840
[13] Z. Chen, J. Zhang, X. Zheng, G. Min, J. Li, C. Rong, Profit-aware cooperative offloading in UAV-enabled MEC systems using lightweight deep reinforcement learning, IEEE Internet Things J. 11 (12) (2024) 21325–21336. https://doi.org/10.1109/JIOT.2023.3331722
[14] R. Zhou, Y. Huang, Y. Wang, L. Jiao, H. Tan, R. Zhang, L. Wu, User preference oriented service caching and task offloading for UAV-assisted MEC networks, IEEE Trans. Serv. Comput. 18 (2) (2025) 1097–1109. https://doi.org/10.1109/TSC.2025.3536319
[15] Y. Zhang, Z. Na, Z. Wen, A. Nallanathan, W. Lu, Joint service caching, computation offloading and resource allocation for dual-layer aerial internet of things, Comput. Netw. 257 (2025) 110974. https://doi.org/10.1016/j.comnet.2024.110974
[16] G. Zheng, C. Xu, M. Wen, X. Zhao, Service caching based aerial cooperative computing and resource allocation in multi-UAV enabled MEC systems, IEEE Trans. Veh. Technol. 71 (10) (2022) 10934–10947. https://doi.org/10.1109/TVT.2022.3183577
[17] L. Wang, Y. Li, Y. Chen, T. Li, Z. Yin, Air-ground coordinated MEC: joint task, time allocation and trajectory design, IEEE Trans. Veh. Technol. 74 (3) (2025) 4728–4743. https://doi.org/10.1109/TVT.2024.3486036
[18] L. Bao, J. Luo, H. Bao, Y. Hao, M. Zhao, Cooperative computation and cache scheduling for UAV-enabled MEC networks, IEEE Trans. Green Commun. Netw. 6 (2) (2022) 965–978. https://doi.org/10.1109/TGCN.2021.3118611
[19] L. Tan, Z. Kuang, L. Zhao, A. Liu, Energy-efficient joint task offloading and resource allocation in OFDMA-based collaborative edge computing, IEEE Trans. Wirel. Commun. 21 (3) (2022) 1960–1972. https://doi.org/10.1109/TWC.2021.3108641
[20] J. Zhou, X. Zhang, Fairness-aware task offloading and resource allocation in cooperative mobile-edge computing, IEEE Internet Things J. 9 (5) (2022) 3812–3824. https://doi.org/10.1109/JIOT.2021.3100253
[21] H. Guo, Y. Wang, J. Liu, C. Liu, Multi-UAV cooperative task offloading and resource allocation in 5G advanced and beyond, IEEE Trans. Wirel. Commun. 23 (1) (2024) 347–359. https://doi.org/10.1109/TWC.2023.3277801
[22] Z. Xu, L. Zhou, H. Dai, W. Liang, W. Zhou, P. Zhou, W. Xu, G. Wu, Energy-aware collaborative service caching in a 5G-enabled MEC with uncertain payoffs, IEEE Trans. Commun. 70 (2) (2022) 1058–1071. https://doi.org/10.1109/TWC.2023.3277801
[23] J. Chen, H. Xing, X. Lin, A. Nallanathan, S. Bi, Joint resource allocation and cache placement for location-aware multi-user mobile-edge computing, IEEE Internet Things J. 9 (24) (2022) 25698–25714. https://doi.org/10.1109/JIOT.2022.3196908
[24] Z. Su, Q. Xu, F. Hou, Q. Yang, Q. Qi, Edge caching for layered video contents in mobile social networks, IEEE Trans. Multimedia 19 (10) (2017) 2210–2221. https://doi.org/10.1109/TMM.2017.2733338
[25] J. Luo, J. Song, F.C. Zheng, L. Gao, T. Wang, User-centric UAV deployment and content placement in cache-enabled multi-UAV networks, IEEE Trans.

Veh. Technol. 71 (5) (2022) 5656–5660. https://doi.org/10.1109/TVT.2022.3152246
[26] S. Taimoor, M. Naeem, S. Zeadally, W. Ejaz, Resource optimization for minimizing latency and cost in UAV-assisted mobile edge computing (MEC) networks, Comput. Netw., 271, 2025, 111587. https://doi.org/10.1016/j.comnet.2025.111587
[27] X. Gao, L. Zhai, Service experience oriented cooperative computing in cache-enabled UAVs assisted MEC networks, IEEE Trans. Mobile Comput. 23 (10) (2024) 9721–9736. https://doi.org/10.1109/TMC.2024.3366944
[28] J. Wang, L. Wang, K. Zhu, P. Dai, Lyapunov-based joint flight trajectory and computation offloading optimization for UAV-assisted vehicular networks, IEEE Internet Things J. 11 (12) (2024) 22243–22256. https://doi.org/10.1109/JIOT.2024.3382242
[29] D. Wang, Y. Jia, L. Liang, K. Ota, M. Dong, Resource allocation in blockchain integration of UAV-enabled MEC networks: a stackelberg differential game approach, IEEE Trans. Serv. Comput. 17 (6) (2024) 4197–4210. https://doi.org/10.1109/TSC.2024.3418330
[30] Z. Chen, Y. Yang, J. Xu, Y. Chen, J. Huang, Task offloading and resource pricing based on game theory in UAV-assisted edge computing, IEEE Trans. Serv. Comput. 18 (1) (2025) 440–452. https://doi.org/10.1109/TSC.2024.3512936
[31] G. Sun, Y. Wang, Z. Sun, Q. Wu, J. Kang, D. Niyato, V.C.M. Leung, Multi-objective optimization for multi-UAV-assisted mobile edge computing, IEEE Trans. Mobile Comput. 23 (12) (2024a) 14803–14820. https://doi.org/10.1109/TMC.2024.3446819
[32] G. Sun, L. He, Z. Sun, Q. Wu, S. Liang, J. Li, D. Niyato, V.C.M. Leung, Joint task offloading and resource allocation in aerial-terrestrial UAV networks with edge and fog computing for post-disaster rescue, IEEE Trans. Mobile Comput. 23 (9) (2024b) 8582–8600. https://doi.org/10.1109/TMC.2024.3350886
[33] M.F. Dehkordi, B. Jabbari, Joint long-term processed task and communication delay optimization in UAV-assisted MEC systems using DQN, in: 2024 IEEE Military Communications Conference, IEEE, 2024, pp. 294–299. https://doi.org/10.1109/MILCOM61039.2024.10773951
[34] M. Darchini-Tabrizi, A. Pakdaman-Donyavi, R. Entezari-Maleki, L. Sousa, Performance enhancement of UAV-enabled MEC systems through intelligent task offloading and resource allocation, Comput. Netw., 264, 2025, 111280. https://doi.org/10.1016/j.comnet.2025.111280
[35] L. Zhao, Y. Yao, J. Guo, Q. Zuo, V.C. Leung, Collaborative computation offloading and wireless charging scheduling in multi-UAV-assisted MEC networks: a TD3-based approach, Comput. Netw., 251, 2024, 110615. https://doi.org/10.1016/j.comnet.2024.110615
[36] A. Mondal, D. Mishra, G. Prasad, H. Johansson, Joint trajectory, user-association, and power control for green UAV-assisted data collection using deep reinforcement learning, IEEE Trans. Intell. Veh. 10 (1) (2025) 548–561. https://doi.org/10.1109/TIV.2024.3416067
[37] L. Zhao, Y. Zhang, S. Zhang, C. Gu, Y. Yao, V.C. Leung, Collaborative offloading and 3D trajectory design in multi-UAV-assisted MEC networks for emergency rescue, in: Proc. Federated Learning and Edge AI for Privacy and Mobility, 2025, pp. 1–3.
[38] S. Wang, X. Song, T. Song, Y. Yang, Fairness-aware computation offloading with trajectory optimization and phase-shift design in RIS-assisted multi-UAV MEC network, IEEE Internet Things J. 11 (11) (2024) 20547–20561. https://doi.org/10.1109/JIOT.2024.3371395
[39] X. Li, X. Du, N. Zhao, X. Wang, Computing over the sky: joint UAV trajectory and task offloading scheme based on optimization-embedding multi-agent deep reinforcement learning, IEEE Trans. Commun. 72 (3) (2024) 1355–1369. https://doi.org/10.1109/TCOMM.2023.3331029
[40] W. Zhang, L. Tan, T. Huang, X. Huang, M. Huang, G. Zhang, Resource allocation and trajectory optimization in multi-UAV collaborative vehicular networks: an extended multiagent DRL approach, IEEE Internet Things J. 12 (8) (2025) 9391–9404. https://doi.org/10.1109/JIOT.2024.3492953
[41] Y. He, K. Xiang, X. Cao, M. Guizani, Task scheduling and trajectory optimization based on fairness and communication security for multi-UAV-MEC system, IEEE Internet Things J. 11 (19) (2024) 30510–30523. https://doi.org/10.1109/JIOT.2024.3412825
[42] H. Hao, C. Xu, W. Zhang, S. Yang, G.M. Muntean, Joint task offloading, resource allocation, and trajectory design for multi-UAV cooperative edge computing with task priority, IEEE Trans. Mobile Comput. 23 (9) (2025) 8649–8663. https://doi.org/10.1109/TMC.2024.3350078
[43] X. He, Y. Zhao, Z. Xu, Y. Chen, Resource allocation strategy for UAV-assisted non-linear energy harvesting MEC system, in: 2022 IEEE 95Th Vehicular Technology Conference: (VTC2022-Spring), IEEE, 2022, pp. 1–7. https://doi.org/10.1109/VTC2022-Spring54318.2022.9860576
[44] S. Müller, O. Atan, M.V.D. Schaar, A. Klein, Context-aware proactive content caching with service differentiation in wireless networks, IEEE Trans. Wirel. Commun. 16 (2) (2017) 1024–1036. https://doi.org/10.1109/TWC.2016.2636139
[45] L. Chen, J. Xu, S. Ren, P. Zhou, Spatio-temporal edge service placement: a bandit learning approach, IEEE Trans. Wirel. Commun. 17 (12) (2018) 8388–8401. https://doi.org/10.1109/TWC.2018.2876823
[46] L. Chen, J. Xu, Budget-constrained edge service provisioning with demand estimation via bandit learning, IEEE J. Sel. Areas Commun. 37 (10) (2019) 2364–2376. https://doi.org/10.1109/JSAC.2019.2933781
[47] H. Ishii, T. Ibaraki, H. Mine, Fractional knapsack problems, Math. Program. 13 (1) (1977) 255–271. https://doi.org/10.1007/BF01584342
[48] D. Monderer, L.S. Shapley, Potential games, Games Econ. Behav. 14 (1) (1996) 124–143. https://doi.org/10.1006/game.1996.0044